

claude-windows / 6e9d0958-a2ec-4a8e-bb2d-6cce9833c527

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\subagents\agent-a069c308d29bb363c.jsonl`
- SHA-256: `0f8d81fa0d8582a4d480d135ee3ba28a54744af359428f15cefbbbc3e569df86`
- Source modified: `2026-07-03T17:36:57+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `subagents`
- session_id: `6e9d0958-a2ec-4a8e-bb2d-6cce9833c527`

Transcript

1. user (2026-07-03T17:33:35.285Z)

You are a code-review finder agent. Review target: PR #452 of badminton-highlight-indexer at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer, branch codex/a2-golden-manifest-portability (HEAD 5a91701). Diff: git diff origin/master..HEAD. The PR introduces backend/eval/golden_manifest.py (load_resolved_manifest with strict=True default; rebases stale paths by basename search across candidate dirs) and rewires manifest loading in: calibrate_local, distill_local, fusion_golden, rally_seg_eval (required_fields=("trajectory",)), rally_seq_proto, serve_contrast, served_gate_a, training/gen0/harness.

ANGLE C ? cross-file tracer. For each changed function, Grep for ALL its callers and check the change doesn't break them:

1. rally_seg_eval.load_golden ? who calls it (scripts/nightly_regression.py, backend/eval/ablation.py?, fusion_compare.py?, tests)? For each caller: does it ever run where manifest exists but a trajectory file is absent (CI, fresh clone, cloud)? Does any caller catch the new ManifestPathError? Does nightly_regression's manifest fingerprint/staleness logic read the manifest separately with RAW paths, and does the rebasing make score-time paths diverge from fingerprint-time paths in a way that matters (baseline invalidation, config hash)?
2. served_gate_a.run / serve_contrast.main ? other entry points (promotion.promote_if_served_safe? tests/test_served_gate_a_regression)?
3. fusion_golden.run ? resumable/--fresh flow and fusion_audio.py: fusion_audio reads the manifest too (does it? grep). If fusion_audio still raw-json.loads the manifest while fusion_golden rebases, the two passes could disagree about which videos exist -> augment mismatch. Same check for backend/eval/fusion_compare.py and backend/eval/ablation.py and backend/eval/experiment.py ? do any OTHER manifest consumers remain on raw json.load (the PR body says it wired 'the flat golden-manifest consumers used by A2 quality work' ? find any left behind that A2 quality work also touches, e.g. scratch/at_par.py, backend/eval/calibrate_wasb.load_golden_gts, gt_loader, windowing tools).
4. golden_manifest.repo_root() ? parents[2]; correct from backend/eval/? And load_resolved_manifest's exceptions: which callers would crash at module-import time (test collection) vs runtime?
5. The docs' smoke command `python -m backend.eval.golden\_manifest` ? does the module guard exist and return proper exit codes for CI use?

Flag only real breakages/misses with concrete scenarios. Return up to 6 candidates as JSON [{file, line, summary, failure_scenario}]. [] if none. Final message = only the JSON array.

2. assistant (2026-07-03T17:33:37.234Z)

I'll analyze PR #452 systematically, checking for cross-file tracer issues. Let me start by understanding the repository structure and the PR changes.

3. user (2026-07-03T17:33:38.221Z)

On branch codex/a2-golden-manifest-portability
Your branch is up to date with 'origin/codex/a2-golden-manifest-portability'.

nothing to commit, working tree clean

4. assistant (2026-07-03T17:33:40.050Z)

Now let me look at the PR diff and the new/modified files:

5. user (2026-07-03T17:33:41.468Z)

```
backend/eval/calibrate_local.py
backend/eval/distill_local.py
backend/eval/fusion_golden.py
backend/eval/golden_manifest.py
backend/eval/rally_seg_eval.py
backend/eval/rally_seq_proto.py
backend/eval/serve_contrast.py
backend/eval/served_gate_a.py
docs/ALPHA_LAUNCH_READINESS.md
docs/data_pipeline/GOLDEN_VIDEOS.md
tests/test_fusion_compare.py
tests/test_fusion_golden_telemetry.py
tests/test_golden_manifest.py
tests/test_served_gate_a_regression.py
training/gen0/harness.py
```

6. assistant (2026-07-03T17:33:43.238Z)

Let me examine the new golden_manifest.py file:

7. user (2026-07-03T17:33:43.568Z)

```
1      """Portable golden-corpus manifest loader and smoke checker.
2
3      The owner-side golden manifest is intentionally gitignored and has historically
4      carried absolute paths from whichever box ingested a clip. This module keeps that
5      artifact usable after the workspace moves by resolving stale paths against the
6      current repo layout and failing loudly when required corpus files are absent.
7
8      Default local layout supported:
9      - ../Annotation Setup/Trajectories/<trajectory>.csv
10     - ../Annotation Setup/Golden Labelled/<labels>.csv
11     - output/<labels>.csv
12
13     Extra roots can be supplied with ``--root`` or ``RALLY_GOLDEN_SEARCH_ROOTS``.
14     """
15
16     from __future__ import annotations
17
18     import argparse
19     import json
20     import os
21     import sys
22     from dataclasses import dataclass
23     from pathlib import Path, PureWindowsPath
24     from typing import Any, Iterable, Sequence
25
26     DEFAULT_MANIFEST = "output/human_lovo_manifest.json"
27     SEARCH_ROOTS_ENV = "RALLY_GOLDEN_SEARCH_ROOTS"
28     DEFAULT_REQUIRED_FIELDS = ("trajectory", "labels")
29
30     _FIELD_DIR = {
31         "trajectory": "Trajectories",
32         "labels": "Golden Labelled",
33     }
34
```

```

35
36 @dataclass(frozen=True)
37 class ManifestIssue:
38     """One manifest path problem found by the resolver."""
39
40     name: str
41     field: str
42     path: str
43     reason: str
44
45     def message(self) -> str:
46         shown = f" {self.path!r}" if self.path else ""
47         return f"{self.name}: {self.field}{shown} - {self.reason}"
48
49
50 class ManifestPathError(RuntimeError):
51     """Raised when a strict manifest load finds missing required files."""
52
53     def __init__(self, manifest_path: str | os.PathLike[str], issues:
Sequence[ManifestIssue]):
54         self.manifest_path = os.fspath(manifest_path)
55         self.issues = list(issues)
56         detail = "\n".join(f" - {i.message()}" for i in self.issues[:12])
57         more = "" if len(self.issues) <= 12 else f"\n ... {len(self.issues) - 12} more"
58         super().__init__(
59             f"golden manifest path check failed for {self.manifest_path}: "
60             f"{len(self.issues)} issue(s)\n{detail}{more}"
61         )
62
63
64 def repo_root() -> Path:
65     """Return the repository root for this module."""
66
67     return Path(__file__).resolve().parents[2]
68
69
70 def load_manifest(manifest_path: str | os.PathLike[str]) -> list[dict[str, Any]]:
71     """Load the flat golden manifest list and validate its outer shape."""
72
73     path = Path(manifest_path)
74     with path.open(encoding="utf-8") as f:
75         data = json.load(f)
76     if not isinstance(data, list):
77         raise ValueError(f"golden manifest must be a JSON list: {path}")
78     out: list[dict[str, Any]] = []
79     for i, row in enumerate(data):
80         if not isinstance(row, dict):
81             raise ValueError(f"golden manifest row {i} is not an object: {path}")
82         out.append(dict(row))
83     return out
84
85
86 def resolve_manifest_paths(
87     manifest_path: str | os.PathLike[str],
88     *,
89     repo_root_path: str | os.PathLike[str] | None = None,
90     extra_roots: Iterable[str | os.PathLike[str]] = (),
91     required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
92 ) -> tuple[list[dict[str, Any]], list[ManifestIssue]]:
93     """Load and rebase required path fields in a golden manifest.
94
95     Returns ``(resolved_specs, issues)``. The returned specs preserve all manifest
96     metadata but replace any resolved ``trajectory`` / ``labels`` path with the
97     existing local path. Missing required fields are reported in ``issues``.
98     """

```

```

99     manifest = load_manifest(manifest_path)
100     return resolve_manifest_specs(
101         manifest,
102         manifest_path,
103         repo_root_path=repo_root_path,
104         extra_roots=extra_roots,
105         required_fields=required_fields,
106     )
107
108
109
110 def load_resolved_manifest(
111     manifest_path: str | os.PathLike[str],
112     *,
113     repo_root_path: str | os.PathLike[str] | None = None,
114     extra_roots: Iterable[str | os.PathLike[str]] = (),
115     required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
116     strict: bool = True,
117 ) -> list[dict[str, Any]]:
118     """Load a manifest with stale paths rebased; optionally raise on missing files."""
119
120     specs, issues = resolve_manifest_paths(
121         manifest_path,
122         repo_root_path=repo_root_path,
123         extra_roots=extra_roots,
124         required_fields=required_fields,
125     )
126     if strict and issues:
127         raise ManifestPathError(manifest_path, issues)
128     return specs
129
130
131 def resolve_manifest_specs(
132     specs: Sequence[dict[str, Any]],
133     manifest_path: str | os.PathLike[str],
134     *,
135     repo_root_path: str | os.PathLike[str] | None = None,
136     extra_roots: Iterable[str | os.PathLike[str]] = (),
137     required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
138 ) -> tuple[list[dict[str, Any]], list[ManifestIssue]]:
139     """Resolve an already-loaded manifest list."""
140
141     manifest = Path(manifest_path)
142     root = Path(repo_root_path).resolve() if repo_root_path else repo_root()
143     roots = [Path(p) for p in _env_roots()]
144     roots.extend(Path(p) for p in extra_roots)
145     required = tuple(required_fields)
146
147     resolved: list[dict[str, Any]] = []
148     issues: list[ManifestIssue] = []
149     for idx, spec in enumerate(specs):
150         row = dict(spec)
151         name = str(row.get("name") or f"row[{idx}]")
152         for field in required:
153             raw = row.get(field)
154             if not raw:
155                 issues.append(ManifestIssue(name, field, "", "required field missing"))
156                 continue
157             hit = resolve_existing_path(
158                 str(raw),
159                 field,
160                 manifest_path=manifest,
161                 repo_root_path=root,
162                 extra_roots=roots,
163             )

```

```

164         if hit is None:
165             issues.append(ManifestIssue(name, field, str(raw), "file not found"))
166         else:
167             row[field] = str(hit)
168             resolved.append(row)
169     return resolved, issues
170
171
172 def resolve_existing_path(
173     raw_path: str,
174     field: str,
175     *,
176     manifest_path: str | os.PathLike[str],
177     repo_root_path: str | os.PathLike[str],
178     extra_roots: Iterable[str | os.PathLike[str]] = (),
179 ) -> Path | None:
180     """Resolve one manifest path field to an existing file, if possible."""
181
182     for candidate in _candidate_paths(
183         raw_path,
184         field,
185         Path(manifest_path),
186         Path(repo_root_path),
187         [Path(p) for p in extra_roots],
188     ):
189         if candidate.is_file():
190             return candidate.resolve()
191     return None
192
193
194 def _env_roots() -> list[Path]:
195     raw = os.environ.get(SEARCH_ROOTS_ENV, "")
196     if not raw:
197         return []
198     return [Path(p) for p in raw.split(os.pathsep) if p.strip()]
199
200
201 def _basename(raw_path: str) -> str:
202     # PureWindowsPath handles stale Windows paths even when tests run on POSIX.
203     win_name = PureWindowsPath(raw_path).name
204     return win_name or Path(raw_path).name
205
206
207 def _candidate_paths(
208     raw_path: str,
209     field: str,
210     manifest_path: Path,
211     root: Path,
212     extra_roots: Sequence[Path],
213 ) -> list[Path]:
214     basename = _basename(raw_path)
215     raw = Path(os.path.expandvars(os.path.expanduser(raw_path)))
216     manifest_dir = manifest_path.resolve().parent
217     candidates: list[Path] = [raw]
218
219     if not raw.is_absolute():
220         candidates.extend([manifest_dir / raw, root / raw])
221
222     for directory in _candidate_dirs(field, manifest_dir, root, extra_roots):
223         candidates.append(directory / basename)
224     return _dedup_paths(candidates)
225
226
227 def _candidate_dirs(
228     field: str, manifest_dir: Path, root: Path, extra_roots: Sequence[Path]

```

```

229 ) -> list[Path]:
230     field_dir = _FIELD_DIR.get(field)
231     dirs: list[Path] = [manifest_dir]
232     if field_dir:
233         dirs.extend(
234             [
235                 root.parent / "Annotation Setup" / field_dir,
236                 root.parent / "Annotation Setup" / "Collect" / field_dir,
237                 root / "output",
238             ]
239         )
240     dirs.extend(root_dirs for r in extra_roots for root_dirs in _expand_root(r, field))
241     return _dedup_paths(dirs)
242
243
244 def _expand_root(root: Path, field: str) -> list[Path]:
245     field_dir = _FIELD_DIR.get(field)
246     dirs = [root, root / "output"]
247     if field_dir:
248         dirs.extend(
249             [
250                 root / field_dir,
251                 root / "Annotation Setup" / field_dir,
252                 root / "Annotation Setup" / "Collect" / field_dir,
253             ]
254         )
255     return dirs
256
257
258 def _dedup_paths(paths: Iterable[Path]) -> list[Path]:
259     out: list[Path] = []
260     seen: set[str] = set()
261     for path in paths:
262         try:
263             key = os.path.normcase(os.path.abspath(os.fspath(path)))
264         except OSError:
265             key = os.fspath(path)
266         if key in seen:
267             continue
268         seen.add(key)
269         out.append(path)
270     return out
271
272
273 def write_manifest(path: str | os.PathLike[str], specs: Sequence[dict[str, Any]]) ->
None:
274     """Write a resolved manifest JSON atomically enough for local tooling."""
275
276     out = Path(path)
277     out.parent.mkdir(parents=True, exist_ok=True)
278     tmp = out.with_name(out.name + ".tmp")
279     with tmp.open("w", encoding="utf-8") as f:
280         json.dump(list(specs), f, indent=2)
281         f.write("\n")
282     os.replace(tmp, out)
283
284
285 def build_parser() -> argparse.ArgumentParser:
286     p = argparse.ArgumentParser(
287         description="Check/rebase the local golden corpus manifest paths."
288     )
289     p.add_argument("--manifest", default=DEFAULT_MANIFEST)
290     p.add_argument(
291         "--repo-root",
292         default=None,

```

```

293         help="repo root used for default sibling output/Annotation Setup lookups",
294     )
295     p.add_argument(
296         "--root",
297         action="append",
298         default=[],
299         help=(
300             "additional search root; repeatable. Also accepts roots via "
301             f"{SEARCH_ROOTS_ENV} separated by os.pathsep."
302         ),
303     )
304     p.add_argument(
305         "--require",
306         action="append",
307         choices=sorted(_FIELD_DIR),
308         default=None,
309         help="path field to require; repeatable. Defaults to trajectory and labels.",
310     )
311     p.add_argument(
312         "--min-count",
313         type=int,
314         default=None,
315         help="fail unless at least this many manifest rows are present",
316     )
317     p.add_argument(
318         "--write",
319         default=None,
320         help="optional path to write the resolved local manifest JSON",
321     )
322     return p
323
324
325 def main(argv: Sequence[str] | None = None) -> int:
326     args = build_parser().parse_args(argv)
327     required = tuple(args.require or DEFAULT_REQUIRED_FIELDS)
328     try:
329         original = load_manifest(args.manifest)
330         resolved, issues = resolve_manifest_specs(
331             original,
332             args.manifest,
333             repo_root_path=args.repo_root,
334             extra_roots=args.root,
335             required_fields=required,
336         )
337     except (OSError, ValueError, json.JSONDecodeError) as e:
338         print(f"ERROR: {e}", file=sys.stderr)
339         return 2
340
341     if args.min_count is not None and len(resolved) < args.min_count:
342         issues.append(
343             ManifestIssue(
344                 "manifest",
345                 "rows",
346                 str(len(resolved)),
347                 f"expected at least {args.min_count} row(s)",
348             )
349         )
350
351     if issues:
352         print(
353             f"ERROR: golden manifest path check failed ({len(issues)} issue(s)):",
354             file=sys.stderr,
355         )
356         for issue in issues:
357             print(f" - {issue.message()}", file=sys.stderr)

```

```

358         return 2
359
360     rebased = sum(
361         1
362         for before, after in zip(original, resolved)
363         for field in required
364         if str(before.get(field, "")) != str(after.get(field, ""))
365     )
366     print(
367         f"OK: {args.manifest} has {len(resolved)} video(s); "
368         f"{len(resolved) * len(required)} required path(s) present; "
369         f"{rebased} path(s) rebased."
370     )
371     if args.write:
372         write_manifest(args.write, resolved)
373         print(f"wrote resolved manifest: {args.write}")
374     return 0
375
376
377 if __name__ == "__main__":
378     raise SystemExit(main())
379

```

8. assistant (2026-07-03T17:33:45.601Z)

Now let me check the modified files to see how `load_resolved_manifest` is being called:

9. user (2026-07-03T17:33:47.203Z)

```

diff --git a/backend/eval/rally_seg_eval.py b/backend/eval/rally_seg_eval.py
index 0acca2d..3846ce0 100644
--- a/backend/eval/rally_seg_eval.py
+++ b/backend/eval/rally_seg_eval.py
@@ -27,6 +27,7 @@ import os
     from typing import Any, Dict, List, Optional, Tuple

     from backend.eval import eval_stats, metrics, segmentation_metrics, windowing
+from backend.eval.golden_manifest import load_resolved_manifest
     from backend.eval.metrics import Interval
     from backend.eval.windowing import PRESETS, SERVED, WindowingPreset

@@ -185,8 +186,7 @@ def load_golden(
 ) -> List[Dict[str, Any]]:
     """Load each golden video's trajectory path + fps + frame_width (manifest) and GT rally
     intervals (`<name>_golden_features.json` ``gts``). Videos with no GTs are kept but
     flagged."""
-    with open(manifest_path) as fh:
-        manifest = json.load(fh)
+    manifest = load_resolved_manifest(manifest_path, required_fields=("trajectory",))
     out: List[Dict[str, Any]] = []
     for v in manifest:
         feat = os.path.join(features_dir, f"{v['name']}_golden_features.json")

```

10. assistant (2026-07-03T17:33:49.046Z)

Good. Let me continue checking all the modified files and their callers:

11. user (2026-07-03T17:33:50.173Z)

```

diff --git a/backend/eval/serve_contrast.py b/backend/eval/serve_contrast.py
index 328857b..0f1b1f4 100644
--- a/backend/eval/serve_contrast.py
+++ b/backend/eval/serve_contrast.py
@@ -32,6 +32,7 @@ from typing import Any, Dict, List, Sequence, Tuple

```



```

from backend.eval import windowing as W
from backend.eval.calibrate_wasb import load_golden_gts
from backend.eval.gemini_refine import merge_overlaps
+from backend.eval.golden_manifest import load_resolved_manifest
from backend.eval.metrics import score
from backend.eval.promotion import promote_if_served_safe
from backend.pipeline.detectors import rally_gate
@@ -194,8 +195,7 @@ def main() -> None:
    )
    args = ap.parse_args()

-    with open(args.manifest, encoding="utf-8") as f:
-        specs = json.load(f)
+    specs = load_resolved_manifest(args.manifest)
    result = run_contrast(
        specs,
        iou=args.iou,

```

12. user (2026-07-03T17:33:52.989Z)

```

diff --git a/backend/eval/served_gate_a.py b/backend/eval/served_gate_a.py
index 994b9c1..c2151fb 100644
--- a/backend/eval/served_gate_a.py
+++ b/backend/eval/served_gate_a.py
@@ -23,13 +23,13 @@ Run (owner box, golden trajectories present):
    from __future__ import annotations

    import argparse
-    import json
    from dataclasses import dataclass
    from typing import Any, Dict, List, Optional, Tuple

    from backend.config.models import FusionConfig, IndexingConfig, WasbIndexConfig
    from backend.eval import segmentation_metrics as _segm
    from backend.eval.calibrate_wasb import load_golden_gts
+from backend.eval.golden_manifest import load_resolved_manifest
    from backend.eval.metrics import Interval, Score, score
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
    from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
@@ -196,8 +196,7 @@ def format_table(
    def run(
        manifest_path: str, iou: float = 0.5
    ) -> Tuple[List[ServedVideoResult], Dict[str, Any]]:
-        with open(manifest_path, encoding="utf-8") as f:
-            specs = json.load(f)
+        specs = load_resolved_manifest(manifest_path)
        results = evaluate_manifest(specs, iou=iou)
        return results, aggregate(results)

```

13. user (2026-07-03T17:33:56.357Z)

```

diff --git a/backend/eval/fusion_golden.py b/backend/eval/fusion_golden.py
index dd3f6a1..aee6d7e 100644
--- a/backend/eval/fusion_golden.py
+++ b/backend/eval/fusion_golden.py
@@ -25,6 +25,7 @@ from typing import Any, Dict, List, Optional

    from backend.eval import distill_local
    from backend.eval.calibrate_wasb import load_golden_gts
+from backend.eval.golden_manifest import load_resolved_manifest
    from backend.eval.training import label_candidates
    from backend.pipeline.detectors import fusion_features as ff
    from backend.pipeline.detectors import rally_gate
@@ -160,8 +161,7 @@ def run(
    fresh: bool = False,

```

```

    smallest_first: bool = False,
) -> List[str]:
-   with open(manifest_path, encoding="utf-8") as f:
-       specs = json.load(f)
+   specs = load_resolved_manifest(manifest_path)
    if smallest_first:
        # Process small videos first for quick feedback (the slow full match lands last);
        # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).

```

14. assistant (2026-07-03T17:33:59.278Z)

Now let me search for all callers of these modified functions:

15. user (2026-07-03T17:34:00.235Z)

Found 5 files
tests\test_rally_seg_eval.py
tests\test_rally_seg_eval_report.py
docs\archives\quality-iterations\2026-06-18-lovo-resweep-n13\lovo_resweep.py
scripts\nightly_reelcount.py
scripts\nightly_regression.py

16. user (2026-07-03T17:34:02.758Z)

Found 1 file
tests\test_served_gate_a_regression.py

17. user (2026-07-03T17:34:04.953Z)

Found 4 files
tests\test_fusion_compare.py
tests\test_fusion_golden_telemetry.py
backend\eval\fusion_audio.py
backend\eval\fusion_compare.py

18. assistant (2026-07-03T17:34:06.585Z)

Now let me check the critical callers:

19. user (2026-07-03T17:34:07.158Z)

```

1      """Nightly rally-quality regression tripwire.
2
3      Re-scores the **golden corpus** under HEAD via :mod:`backend.eval.rally_seg_eval` (CPU,
4      seconds ? the WASB GPU trajectories are CACHED, only the windowing/scorer changes) and
diffs
5      the headline metrics against a FROZEN baseline
(``eval_baselines/nightly_baseline.json``).
6      Writes a dated report and exits non-zero on any regression beyond tolerance OR any per-
video
7      over-segmentation-guard regression.
8
9      This is the owner's dual-eval-set discipline made automatic (``memory/eval-dual-set-
regression``
10     + ``docs/R2_EVAL_METRIC_DESIGN.md`` §4.5): the growing GOLDEN-finalized set is re-run
every
11     night so a *late* change cannot silently lose an *earlier* win. It is **default-
ADDITIVE** ? it
12     does NOT change the promotion gate. It is a *tripwire*, not a re-training run: no GPU,
no Gemini,
13     no WASB re-extraction (those are cached and unchanged night to night).
14
15     Three tracked configs (all re-scored every night; a clean A/B/C on the same SERVED base

```

```

knobs):
16
17     * **DEFAULT** ? the SERVED production windowing, i.e. what users get today. Catches a
regression
18     to shipped behaviour. **NOTE (post-#204, 2026-06-18):** the R1 milestone is now
flipped ON, so
19     SERVED *is* cap=6 + R4 ? "default" now equals "milestone".
20     * **MILESTONE** ? the explicit R3+R4 config (cap=6 s + rally-END inactivity trim;
21     ``docs/QUALITY_ITERATIONS.md`` R3/R4). Now == DEFAULT (production shipped it); kept
explicit so
22     the nightly still names the milestone knobs and stays meaningful if production later
diverges.
23     * **BASELINE_OFF** ? the pre-R1-flip all-OFF windowing (every quality knob forced OFF).
The
24     permanent historical floor, so each report keeps showing the delta the milestone buys
(a launched
25     win must keep standing its ground, not be taken at launch-time value).
26
27     Ground truth + trajectories are LOCAL (the manifest points at owned, minors-free
trajectory CSVs
28     outside the repo + ``output/<name>_golden_features.json``); the corpus is NOT committed.
So the
29     baseline is tied to the owner's local golden corpus and is regenerated with ``--update-
baseline``
30     whenever the corpus grows or an intended improvement lands. (A cloud agent does not
suffice ? it
31     has the repo but not the cached trajectories; hence the local Windows Scheduled Task
wiring in
32     ``scripts/nightly_regression.ps1``.)
33
34     Usage
35     -----
36     Snapshot the baseline from current HEAD (after an intended improvement or a corpus
change):
37
38     python scripts/nightly_regression.py --update-baseline
39
40     Nightly check (exit 1 on regression; writes ``output/nightly_regression/<date>.md`` +
``.json``):
41
42     python scripts/nightly_regression.py
43
44     Exit codes
45     -----
46     * ``0`` ? all tracked configs clean (no regression), corpus matches the baseline.
47     * ``1`` ? at least one regression (an aggregate metric dropped beyond tolerance, or a
per-video
48     over-seg guard increased). Takes precedence over a stale baseline.
49     * ``2`` ? operational error (manifest/corpus missing, nothing scored).
50     * ``3`` ? no baseline to compare against (snapshot one with ``--update-baseline``).
51     * ``4`` ? baseline STALE (the corpus set or a config definition changed): a refresh is
needed,
52     but no regression was found on the comparable intersection.
53     ""
54
55     from __future__ import annotations
56
57     import argparse
58     import datetime as _dt
59     import json
60     import os
61     import sys
62     from dataclasses import dataclass, field, replace
63     from typing import Any, Dict, List, Optional, Tuple
64

```

```

65 # ----- #
66 # Paths (resolved relative to the repo root, so the script is cwd-robust).
67 # ----- #
68 REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
69 if REPO_ROOT not in sys.path:
70     sys.path.insert(
71         0, REPO_ROOT
72     ) # importable as `python scripts/nightly_regression.py` from anywhere
73 DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "nightly_baseline.json")
74 DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_regression")
75 DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "output", "human_lovo_manifest.json")
76 DEFAULT_FEATURES_DIR = os.path.join(REPO_ROOT, "output")
77 DEFAULT_IOU = 0.5
78
79 # ----- #
80 # What we gate on (a tripwire ? focused, not every metric).
81 # * HIGHER-better aggregates regress when they DROP beyond `metric_tol`.
82 # * merge_rate (lower-better) regresses when it RISES beyond `rate_tol`.
83 # * per-video split_count / merge_count must NOT increase beyond `over_seg_tol`
84 # (the R2 §2.3.3 guard: "may not increase EITHER on ANY video").
85 # Boundary-error medians + seg_ratio are outlier-dominated diagnostics ? REPORTED, not
gated.
86 # ----- #
87 GATED_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
88 GATED_LOWER: Tuple[str, ...] = ("merge_rate",)
89 PER_VIDEO_GUARD: Tuple[str, ...] = ("split_count", "merge_count")
90
91 #: Aggregate metrics snapshotted into the baseline (the gated subset + report-only
context).
92 AGG_SNAPSHOT: Tuple[str, ...] = (
93     "tol_f1",
94     "tol_precision",
95     "tol_recall",
96     "f1@0.5",
97     "f1@0.25",
98     "precision",
99     "recall",
100     "merge_rate",
101     "split_rate",
102     "segment_count_ratio",
103     "dstart_abs_median",
104     "dend_abs_median",
105 )
106 #: Per-video metrics snapshotted into the baseline.
107 PV_SNAPSHOT: Tuple[str, ...] = (
108     "tol_f1",
109     "f1@0.5",
110     "f1@0.25",
111     "merge_rate",
112     "split_count",
113     "merge_count",
114     "num_pred",
115     "num_gt",
116 )
117
118 DEFAULT_METRIC_TOL = (
119     0.005 # F1-like drop that counts as a regression (absorbs float jitter)
120 )
121 DEFAULT_RATE_TOL = 0.010 # merge_rate rise that counts as a regression
122 DEFAULT_OVER_SEG_TOL = 0 # per-video split/merge increase allowed (strict)
123
124
125 # ----- #
126 # The two tracked configs.
127 # ----- #

```

```

128     @dataclass(frozen=True)
129     class NightlyConfig:
130         """A named windowing config the nightly re-scores (preset + net-crossings gate)."""
131
132         name: str
133         preset: Any # backend.eval.windowing.WindowingPreset
134         min_crossings: int = 0
135         note: str = ""
136
137
138     def tracked_configs() -> List[NightlyConfig]:
139         """DEFAULT (SERVED = shipped) + MILESTONE (explicit cap=6 + R4) + BASELINE_OFF (the
140 pre-flip floor).
141
142         All three are anchored on the SAME ``windowing.SERVED`` base knobs (velocity_thresh
143 / merge_gap /
144 min_window) so they differ ONLY in the quality knobs ? they are a clean A/B/C:
145
146 * **DEFAULT** ? ``SERVED`` verbatim, so it can never silently drift from the served
147 operating
148 point. **Post-#204 (2026-06-18) the R1 milestone is flipped ON ? SERVED IS cap=6 +
149 R4, so
150 DEFAULT now equals MILESTONE.** Its job: catch an *accidental* change to a
151 production windowing
152 knob.
153 * **MILESTONE** ? SERVED + the explicit R3/R4 knobs (``docs/QUALITY_ITERATIONS.md``
154 R3/R4). Pins
155 the milestone *definition* independent of what production ships, so "is cap=6 + R4
156 still scoring
157 what we expect?" stays answerable even if a future change moves DEFAULT.
158 * **BASELINE_OFF** ? SERVED with every quality knob forced OFF (the pre-R1-flip
159 windowing). Pins
160 the *historical floor* so each nightly report keeps showing the delta the
161 milestone buys (the
162 dual-eval-set discipline: a launched win must keep standing its ground, not be
163 taken at
164 launch-time value). It is the reference DEFAULT used to be before the flip."""
165 from backend.eval.windowing import SERVED
166
167     default = NightlyConfig(
168         name="default",
169         preset=SERVED,
170         min_crossings=0,
171         note="SERVED production windowing ? now the R1 milestone (cap=6 + R4) after the
172 #204 flip.",
173     )
174     milestone = NightlyConfig(
175         name="milestone",
176         preset=replace(
177             SERVED,
178             name="milestone",
179             max_window_duration=6.0,
180             inactivity_end=1.5,
181             inactivity_rally_thresh=0.20,
182             inactivity_min_density=0.30,
183         ),
184         min_crossings=0,
185         note="R3 cap=6 s + R4 rally-END inactivity trim ? the shipped milestone (==
186 default post-#204).",
187     )
188     baseline_off = NightlyConfig(
189         name="baseline_off",
190         preset=replace(
191             SERVED,
192             name="baseline_off",

```

```

181         max_window_duration=0.0,
182         hard_cap=False,
183         inactivity_end=0.0,
184         inactivity_rally_thresh=0.08,
185         inactivity_min_density=0.34,
186         blob_fallback=False,
187         blob_factor=4.0,
188     ),
189     min_crossings=0,
190     note="Pre-R1-flip all-OFF windowing ? the historical floor; pins the value the
milestone buys.",
191 )
192     return [default, milestone, baseline_off]
193
194
195 # ----- #
196 # Tolerances bundle.
197 # ----- #
198 @dataclass(frozen=True)
199 class Tolerances:
200     metric_tol: float = DEFAULT_METRIC_TOL
201     rate_tol: float = DEFAULT_RATE_TOL
202     over_seg_tol: int = DEFAULT_OVER_SEG_TOL
203
204     def to_dict(self) -> Dict[str, Any]:
205         return {
206             "metric_tol": self.metric_tol,
207             "rate_tol": self.rate_tol,
208             "over_seg_tol": self.over_seg_tol,
209         }
210
211
212 # ----- #
213 # Pure summary + comparison core (no I/O ? unit-testable).
214 # ----- #
215 def summarize_run(eval_result: Dict[str, Any]) -> Dict[str, Any]:
216     """Compact, comparable summary of one :func:`rally_seg_eval.evaluate` result.
217
218     Keeps only the deterministic quantities we compare ? aggregate MEANS (not the
bootstrap CI,
219     which is seeded but irrelevant to the gate) and per-video RAW metric values."""
220     agg = eval_result.get("aggregate", {}) or {}
221     aggregate = {
222         k: (agg[k]["mean"] if isinstance(agg.get(k), dict) else None)
223         for k in AGG_SNAPSHOT
224     }
225     per_video = {
226         r["name"]: {k: r[k] for k in PV_SNAPSHOT if k in r}
227         for r in eval_result.get("rows", [])
228     }
229     return {
230         "n": int(agg.get("n", len(eval_result.get("rows", [])))),
231         "preset": eval_result.get("preset"),
232         "min_crossings": eval_result.get("min_crossings", 0),
233         "iou": eval_result.get("iou", DEFAULT_IOU),
234         "aggregate": aggregate,
235         "per_video": per_video,
236     }
237
238
239 @dataclass
240 class Finding:
241     """One comparison outcome line (regression / improvement / stale note)."""
242
243     config: str

```

```

244     scope: str # "aggregate" | "per_video" | "config"
245     metric: str
246     kind: str # "regression" | "improvement" | "stale"
247     baseline: Optional[float] = None
248     current: Optional[float] = None
249     video: Optional[str] = None
250
251     @property
252     def delta(self) -> Optional[float]:
253         if self.baseline is None or self.current is None:
254             return None
255         return self.current - self.baseline
256
257     def message(self) -> str:
258         loc = (
259             f"{self.config}/{self.video}/{self.metric}"
260             if self.video
261             else f"{self.config}/{self.metric}"
262         )
263         if self.kind == "stale":
264             return f"[STALE] {self.config}: {self.metric}"
265         d = self.delta
266         ds = f"{d:+.4f}" if d is not None else "?"
267         b = "?" if self.baseline is None else f"{self.baseline:.4f}"
268         c = "?" if self.current is None else f"{self.current:.4f}"
269         tag = "REGRESSION" if self.kind == "regression" else "improvement"
270         return f"[{tag}] {loc}: {b} ? {c} ({ds})"
271
272
273     @dataclass
274     class ConfigComparison:
275         config: str
276         status: str = "ok" # "ok" | "regression" | "stale"
277         findings: List[Finding] = field(default_factory=list)
278         added_videos: List[str] = field(default_factory=list)
279         removed_videos: List[str] = field(default_factory=list)
280         aggregate_gated: bool = (
281             True # False when preset/corpus changed ? aggregate not comparable
282         )
283
284         @property
285         def regressions(self) -> List[Finding]:
286             return [f for f in self.findings if f.kind == "regression"]
287
288         @property
289         def improvements(self) -> List[Finding]:
290             return [f for f in self.findings if f.kind == "improvement"]
291
292
293     def _preset_comparable(base_preset: Any, cur_preset: Any) -> bool:
294         """Are the two stored preset dicts the same windowing definition? (name is
295         cosmetic)."""
296         if not isinstance(base_preset, dict) or not isinstance(cur_preset, dict):
297             return base_preset == cur_preset
298         keys = set(base_preset) | set(cur_preset)
299         return all(base_preset.get(k) == cur_preset.get(k) for k in keys if k != "name")
300
301     def compare_config(
302         name: str, base_sum: Dict[str, Any], cur_sum: Dict[str, Any], tols: Tolerances
303     ) -> ConfigComparison:
304         """Compare one config's current summary to its baseline summary.
305
306         * If the preset definition or the net-crossings gate changed, the whole config is
307         STALE

```

```

307         (numbers aren't comparable) ? flag it, skip gating.
308     * If the corpus (video set) changed, the aggregate is not comparable (different
videos) ?
309         skip the aggregate gate + flag added/removed, but STILL run the per-video gate on
the
310         INTERSECTION (a code regression on a shared video must still trip the wire).
311     """
312     cmp = ConfigComparison(config=name)
313
314     # 1) Config-definition drift ? stale, not gateable.
315     if not _preset_comparable(
316         base_sum.get("preset"), cur_sum.get("preset")
317     ) or base_sum.get("min_crossings") != cur_sum.get("min_crossings"):
318         cmp.status = "stale"
319         cmp.aggregate_gated = False
320         cmp.findings.append(
321             Finding(config=name, scope="config", metric="preset", kind="stale")
322         )
323     return cmp
324
325     base_pv = base_sum.get("per_video", {})
326     cur_pv = cur_sum.get("per_video", {})
327     base_names, cur_names = set(base_pv), set(cur_pv)
328     cmp.added_videos = sorted(cur_names - base_names)
329     cmp.removed_videos = sorted(base_names - cur_names)
330     corpus_changed = bool(cmp.added_videos or cmp.removed_videos)
331
332     # 2) Aggregate gate (only when the corpus matches ? means over different sets aren't
comparable).
333     if corpus_changed:
334         cmp.aggregate_gated = False
335         cmp.status = "stale"
336         cmp.findings.append(
337             Finding(config=name, scope="config", metric="corpus", kind="stale")
338         )
339     else:
340         base_agg = base_sum.get("aggregate", {})
341         cur_agg = cur_sum.get("aggregate", {})
342         for m in GATED_HIGHER:
343             b, c = base_agg.get(m), cur_agg.get(m)
344             if b is None or c is None:
345                 continue
346             if c < b - tols.metric_tol:
347                 cmp.findings.append(Finding(name, "aggregate", m, "regression", b, c))
348             elif c > b + tols.metric_tol:
349                 cmp.findings.append(Finding(name, "aggregate", m, "improvement", b, c))
350         for m in GATED_LOWER:
351             b, c = base_agg.get(m), cur_agg.get(m)
352             if b is None or c is None:
353                 continue
354             if c > b + tols.rate_tol:
355                 cmp.findings.append(Finding(name, "aggregate", m, "regression", b, c))
356             elif c < b - tols.rate_tol:
357                 cmp.findings.append(Finding(name, "aggregate", m, "improvement", b, c))
358
359     # 3) Per-video over-seg guard on the intersection (always ? catches per-video code
regressions).
360     for vid in sorted(base_names & cur_names):
361         for m in PER_VIDEO_GUARD:
362             b, c = base_pv[vid].get(m), cur_pv[vid].get(m)
363             if b is None or c is None:
364                 continue
365             if c > b + tols.over_seg_tol:
366                 cmp.findings.append(
367                     Finding(

```



```

368         name,
369         "per_video",
370         m,
371         "regression",
372         float(b),
373         float(c),
374         video=vid,
375     )
376 )
377
378 if cmp.regressions:
379     cmp.status = "regression"
380 elif cmp.status != "stale":
381     cmp.status = "ok"
382 return cmp
383
384
385 @dataclass
386 class NightlyResult:
387     comparisons: List[ConfigComparison] = field(default_factory=list)
388     skipped: Dict[str, List[str]] = field(
389         default_factory=dict
390     ) # config -> manifest videos not scored
391
392     @property
393     def has_regression(self) -> bool:
394         return any(c.status == "regression" for c in self.comparisons)
395
396     @property
397     def has_stale(self) -> bool:
398         return any(c.status == "stale" for c in self.comparisons)
399
400     def overall_status(self) -> str:
401         if self.has_regression:
402             return "REGRESSION"
403         if self.has_stale:
404             return "STALE"
405         return "PASS"
406
407     def exit_code(self) -> int:
408         if self.has_regression:
409             return 1
410         if self.has_stale:
411             return 4
412         return 0
413
414
415 def compare_all(
416     baseline: Dict[str, Any],
417     current: Dict[str, Any],
418     tols: Tolerances,
419     skipped: Optional[Dict[str, List[str]]] = None,
420 ) -> NightlyResult:
421     """Compare every tracked config's current summary to the baseline summary."""
422     res = NightlyResult(skipped=skipped or {})
423     base_cfgs = baseline.get("configs", {})
424     cur_cfgs = current.get("configs", {})
425     for name in cur_cfgs:
426         if name not in base_cfgs:
427             cmp = ConfigComparison(config=name, status="stale", aggregate_gated=False)
428             cmp.findings.append(
429                 Finding(
430                     config=name,
431                     scope="config",
432                     metric="missing_in_baseline",

```

```

433         kind="stale",
434     )
435 )
436 res.comparisons.append(cmp)
437 continue
438 res.comparisons.append(
439     compare_config(name, base_cfgs[name], cur_cfgs[name], tols)
440 )
441 return res
442
443
444 # ----- #
445 # Report formatting (pure).
446 # ----- #
447 def _fmt(v: Optional[float]) -> str:
448     return "?" if v is None else f"{v:.4f}"
449
450
451 def format_report(
452     baseline: Dict[str, Any],
453     current: Dict[str, Any],
454     result: NightlyResult,
455     tols: Tolerances,
456     date: str,
457 ) -> str:
458     L: List[str] = []
459     status = result.overall_status()
460     badge = {
461         "PASS": "? PASS",
462         "REGRESSION": "? REGRESSION",
463         "STALE": "? STALE (baseline refresh needed)",
464     }[status]
465     L.append(f"# Nightly rally-quality regression ? {date}")
466     L.append("")
467     L.append(f"***Status: {badge}** . exit code {result.exit_code()}")
468     L.append("")
469     L.append(
470         f"- HEAD: `{current.get('git_commit', '?')}` . baseline: "
471         f"`{baseline.get('git_commit', '?')}` (snapshot {baseline.get('generated_at',
472             '?')})"
473     )
474     L.append(
475         f"- Corpus manifest: `{current.get('manifest', '?')}` . "
476         f"IoU={current.get('iou', DEFAULT_IOU)}"
477     )
478     L.append(
479         f"- Tolerances: metric  $\pm$ {tols.metric_tol}, rate  $\pm$ {tols.rate_tol}, "
480         f"per-video over-seg  $\pm$ {tols.over_seg_tol}"
481     )
482     L.append("")
483
484 # Up-front regression / stale summary.
485 regs = [f for c in result.comparisons for f in c.regressions]
486 if regs:
487     L.append("## ? Regressions")
488     L.append("")
489     for f in regs:
490         L.append(f"- {f.message()}")
491     L.append("")
492
493 if result.has_stale:
494     L.append("## ? Stale / not comparable")
495     L.append("")
496     for c in result.comparisons:
497         if c.status == "stale":
498             if c.added_videos:

```

```

496         L.append(f"- `{c.config}`: corpus added {c.added_videos}")
497     if c.removed_videos:
498         L.append(f"- `{c.config}`: corpus removed {c.removed_videos}")
499     if not c.added_videos and not c.removed_videos:
500         L.append(
501             f"- `{c.config}`: config definition changed vs baseline ? re-
snapshot with --update-baseline"
502         )
503     L.append(
504         "- Action: `python scripts/nightly_regression.py --update-baseline` (after
confirming the change is intended)."
505     )
506     L.append("")
507
508     # Skipped (no silent caps).
509     skipped_any = {k: v for k, v in result.skipped.items() if v}
510     if skipped_any:
511         L.append(
512             "### ? Skipped manifest videos (missing trajectory or GT ? NOT silently
dropped)"
513         )
514         L.append("")
515         for cfg, vids in skipped_any.items():
516             L.append(f"- `{cfg}`: {vids}")
517         L.append("")
518
519     # Per-config detail.
520     cur_cfgs = current.get("configs", {})
521     base_cfgs = baseline.get("configs", {})
522     for cmp in result.comparisons:
523         cur = cur_cfgs.get(cmp.config, {})
524         base = base_cfgs.get(cmp.config, {})
525         L.append(f"### Config `{cmp.config}` ? {cur.get('note', '')}")
526         L.append("")
527         L.append(
528             f"_status: {cmp.status}; n={cur.get('n', '?')} (baseline n={base.get('n',
'?'')})_"
529         )
530         L.append("")
531         cagg = cur.get("aggregate", {})
532         bagg = base.get("aggregate", {})
533         L.append("| metric | baseline | current | ? | gated |")
534         L.append("|---|---|---|---|---|")
535         for m in AGG_SNAPSHOT:
536             b, c = bagg.get(m), cagg.get(m)
537             d = (c - b) if (b is not None and c is not None) else None
538             gated = (
539                 "?"
540                 if (m in GATED_HIGHER or m in GATED_LOWER) and cmp.aggregate_gated
541                 else ""
542             )
543             L.append(f"| {m} | {_fmt(b)} | {_fmt(c)} | {_fmt(d)} | {gated} |")
544         L.append("")
545     # Per-video over-seg guard table.
546     cpv = cur.get("per_video", {})
547     bpv = base.get("per_video", {})
548     names = sorted(set(cpv) | set(bpv))
549     if names:
550         L.append("| video | tolF1 (b?c) | split (b?c) | merge (b?c) |")
551         L.append("|---|---|---|---|")
552         for v in names:
553             cb, cc = bpv.get(v, {}), cpv.get(v, {})
554             tf = f"{_fmt(cb.get('tol_f1'))}?{_fmt(cc.get('tol_f1'))}"
555             sp = f"{cb.get('split_count', '?')}?{cc.get('split_count', '?')}"
556             mg = f"{cb.get('merge_count', '?')}?{cc.get('merge_count', '?')}"

```

```

557         flag = (
558             " ?"
559             if any(
560                 f.video == v and f.kind == "regression" for f in cmp.findings
561             )
562             else ""
563         )
564         L.append(f"| {v}| {flag} | {tf} | {sp} | {mg} |")
565     L.append("")
566     imps = cmp.improvements
567     if imps:
568         L.append(
569             "_Improvements over baseline (consider `--update-baseline` to ratchet
the floor up):_"
570         )
571         for f in imps:
572             L.append(f"- {f.message()}")
573         L.append("")
574
575     L.append("---")
576     L.append(
577         "_Tripwire only ? re-scores CACHED trajectories (no GPU/Gemini/WASB). Does not
change "
578         "the promotion gate. See docs/R2_EVAL_METRIC_DESIGN.md + memory/eval-dual-set-
regression._"
579     )
580     return "\n".join(L)
581
582
583     # ----- #
584     # I/O shell.
585     # ----- #
586     def _git_commit() -> str:
587         import subprocess
588
589         try:
590             out = subprocess.run(
591                 ["git", "-C", REPO_ROOT, "rev-parse", "--short", "HEAD"],
592                 capture_output=True,
593                 text=True,
594                 timeout=10,
595             )
596             return out.stdout.strip() or "?"
597         except Exception:
598             return "?"
599
600
601     def _display_path(path: str) -> str:
602         """A clean, portable form for the committed baseline: ``output/<file>`` when the
path lives
603         in an ``output`` dir (the production layout), else repo-relative, else the path as
given.
604         Metadata only ? the comparison never reads it."""
605         norm = path.replace("\\", "/")
606         parent = os.path.basename(os.path.dirname(norm))
607         if parent == "output":
608             return "output/" + os.path.basename(norm)
609         try:
610             rel = os.path.relpath(path, REPO_ROOT)
611             if not rel.startswith(".."):
612                 return rel.replace("\\", "/")
613         except ValueError:
614             pass
615         return norm
616

```

```

617
618     def run_corpus(
619         configs: List[NightlyConfig],
620         manifest: str,
621         features_dir: str,
622         iou: float = DEFAULT_IUO,
623     ) -> Tuple[Dict[str, Any], Dict[str, List[str]]]:
624         """Score every tracked config over the golden corpus. Returns (snapshot, skipped-
625         per-config).
626         ``snapshot`` is the serialisable structure stored as / compared to the baseline.
627         ``skipped``
628         lists manifest videos NOT scored (missing trajectory or empty GT) per config ?
629         surfaced in
630         the report so the corpus can't silently shrink."""
631         from backend.eval import rally_seg_eval
632
633         golden = rally_seg_eval.load_golden(manifest, features_dir)
634         manifest_names = [v["name"] for v in golden]
635         if not golden:
636             raise RuntimeError(f"no golden videos loaded from {manifest}")
637
638         snapshot: Dict[str, Any] = {
639             "manifest": _display_path(manifest),
640             "iou": iou,
641             "configs": {},
642         }
643         skipped: Dict[str, List[str]] = {}
644         scored_any = False
645         for cfg in configs:
646             result = rally_seg_eval.evaluate(
647                 golden, cfg.preset, iou=iou, min_crossings=cfg.min_crossings
648             )
649             summary = summarize_run(result)
650             summary["note"] = cfg.note
651             snapshot["configs"][cfg.name] = summary
652             scored = set(summary["per_video"])
653             skipped[cfg.name] = [n for n in manifest_names if n not in scored]
654             scored_any = scored_any or bool(scored)
655         if not scored_any:
656             raise RuntimeError(
657                 f"scored 0 videos for every config ? are the trajectory CSVs in {manifest}
658                 present "
659                 f"and do the {features_dir}/<name>_golden_features.json files have 'gts'?"
660             )
661         return snapshot, skipped
662
663     def load_baseline(path: str) -> Optional[Dict[str, Any]]:
664         if not os.path.exists(path):
665             return None
666         with open(path, encoding="utf-8") as fh:
667             return json.load(fh)
668
669     def save_baseline(path: str, snapshot: Dict[str, Any], tols: Tolerances) -> None:
670         out = dict(snapshot)
671         out["generated_at"] = _dt.date.today().isoformat()
672         out["git_commit"] = _git_commit()
673         out["tolerances"] = tols.to_dict()
674         out["_doc"] = (
675             "Frozen nightly rally-quality regression baseline. Regenerate with "
676             "`python scripts/nightly_regression.py --update-baseline` after an intended "
677             "improvement or a corpus change. Tied to the LOCAL golden corpus (not
678             committed)."
679         )

```

```

677     )
678     os.makedirs(os.path.dirname(path), exist_ok=True)
679     tmp = path + ".tmp"
680     with open(tmp, "w", encoding="utf-8") as fh:
681         json.dump(out, fh, indent=2, sort_keys=True)
682         fh.write("\n")
683     os.replace(tmp, path)
684
685
686 def build_parser() -> argparse.ArgumentParser:
687     p = argparse.ArgumentParser(
688         description="Nightly rally-quality regression tripwire (offline, CPU).")
689     )
690     p.add_argument("--manifest", default=DEFAULT_MANIFEST, help="golden manifest JSON")
691     p.add_argument(
692         "--features-dir",
693         default=DEFAULT_FEATURES_DIR,
694         help="dir holding <name>_golden_features.json",
695     )
696     p.add_argument(
697         "--baseline", default=DEFAULT_BASELINE, help="frozen baseline JSON path"
698     )
699     p.add_argument(
700         "--report-dir",
701         default=DEFAULT_REPORT_DIR,
702         help="where dated reports are written",
703     )
704     p.add_argument("--iou", type=float, default=DEFAULT_IOU)
705     p.add_argument(
706         "--metric-tol",
707         type=float,
708         default=DEFAULT_METRIC_TOL,
709         help="F1-like aggregate drop that counts as a regression",
710     )
711     p.add_argument(
712         "--rate-tol",
713         type=float,
714         default=DEFAULT_RATE_TOL,
715         help="merge_rate rise that counts as a regression",
716     )
717     p.add_argument(
718         "--over-seg-tol",
719         type=int,
720         default=DEFAULT_OVER_SEG_TOL,
721         help="per-video split/merge-count increase allowed before it's a regression",
722     )
723     p.add_argument(
724         "--update-baseline",
725         action="store_true",
726         help="snapshot the current numbers as the new baseline instead of checking",
727     )
728     p.add_argument(
729         "--date", default=None, help="override the report date (YYYY-MM-DD; for tests)"
730     )
731     p.add_argument(
732         "--no-report", action="store_true", help="skip writing the dated report file"
733     )
734     return p
735
736
737 def main(argv: Optional[List[str]] = None) -> int:
738     import logging
739
740     logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
741     log = logging.getLogger("nightly_regression")

```

```

742     args = build_parser().parse_args(argv)
743     tols = Tolerances(
744         metric_tol=args.metric_tol,
745         rate_tol=args.rate_tol,
746         over_seg_tol=args.over_seg_tol,
747     )
748     date = args.date or _dt.date.today().isoformat()
749
750     if not os.path.exists(args.manifest):
751         log.error(
752             "manifest not found: %s ? the golden corpus must be present locally.",
753             args.manifest,
754         )
755         return 2
756
757     try:
758         snapshot, skipped = run_corpus(
759             tracked_configs(), args.manifest, args.features_dir, args.iou
760         )
761     except RuntimeError as e:
762         log.error("%s", e)
763         return 2
764     snapshot["git_commit"] = _git_commit()
765     snapshot["generated_at"] = date
766
767     if args.update_baseline:
768         save_baseline(args.baseline, snapshot, tols)
769         log.info("baseline snapshot written to %s", args.baseline)
770         for cfg, s in snapshot["configs"].items():
771             agg = s["aggregate"]
772             log.info(
773                 " %s: n=%s tolF1=%.3f f1@0.5=%.3f merge_rate=%.3f",
774                 cfg,
775                 s["n"],
776                 agg.get("tol_f1") or 0.0,
777                 agg.get("f1@0.5") or 0.0,
778                 agg.get("merge_rate") or 0.0,
779             )
780         return 0
781
782     baseline = load_baseline(args.baseline)
783     if baseline is None:
784         log.error(
785             "no baseline at %s ? snapshot one with --update-baseline (then commit it).",
786             args.baseline,
787         )
788         return 3
789
790     result = compare_all(baseline, snapshot, tols, skipped)
791     report = format_report(baseline, snapshot, result, tols, date)
792
793     if not args.no_report:
794         os.makedirs(args.report_dir, exist_ok=True)
795         md_path = os.path.join(args.report_dir, f"{date}.md")
796         json_path = os.path.join(args.report_dir, f"{date}.json")
797         with open(md_path, "w", encoding="utf-8") as fh:
798             fh.write(report + "\n")
799         with open(json_path, "w", encoding="utf-8") as fh:
800             json.dump(
801                 {
802                     "date": date,
803                     "status": result.overall_status(),
804                     "exit_code": result.exit_code(),
805                     "current": snapshot,
806                     "regressions": [

```

```

807             f.message() for c in result.comparisons for f in c.regressions
808         ],
809     },
810     fh,
811     indent=2,
812     sort_keys=True,
813 )
814     log.info("report written to %s", md_path)
815
816     print(report)
817     code = result.exit_code()
818     if code == 1:
819         log.error(
820             "NIGHTLY REGRESSION DETECTED (%d) ? see %s",
821             sum(len(c.regressions) for c in result.comparisons),
822             args.report_dir,
823         )
824     elif code == 4:
825         log.warning(
826             "baseline STALE ? corpus/config changed; refresh with --update-baseline"
827         )
828     else:
829         log.info("nightly regression check PASSED")
830     return code
831
832
833 if __name__ == "__main__":
834     raise SystemExit(main())
835

```

20. assistant (2026-07-03T17:34:09.383Z)

Now let me check other manifest consumers mentioned in the angle:

21. user (2026-07-03T17:34:11.123Z)

```

backend/eval/golden_fixtures.py:     entries = load_manifest(fixtures_dir)
backend/eval/golden_fixtures.py:def load_manifest(fixtures_dir: Path = FIXTURES_DIR) ->
List[Dict[str, Any]]:
backend/eval/golden_fixtures.py:     manifest = load_manifest(fixtures_dir)
backend/eval/golden_manifest.py:def load_manifest(manifest_path: str | os.PathLike[str]) ->
List[dict[str, Any]]:
backend/eval/golden_manifest.py:     manifest = load_manifest(manifest_path)
backend/eval/golden_manifest.py:         original = load_manifest(args.manifest)

```

22. user (2026-07-03T17:34:18.311Z)

Found 9 files

```

backend\eval\rally_seg_eval.py
backend\eval\golden_manifest.py
backend\eval\per_user_regression.py
backend\eval\regression.py
backend\eval\splits.py
backend\eval\classifier.py
backend\eval\experiment.py
backend\eval\fusion_audio.py
backend\eval\golden_fixtures.py

```

23. assistant (2026-07-03T17:34:20.045Z)

Let me check fusion_audio.py and experiment.py:

24. user (2026-07-03T17:34:20.368Z)


```

1      """Fusion P4 ? measure the INCREMENTAL audio-cue lift on top of the person cue (CPU-
only).
2
3      Reuses the person-feature JSONs from `fusion_golden.py` (no GPU re-detect): `augment`
runs
4      `detect_hits_in_video` once per video (ffmpeg + numpy, CPU) and adds the 3 audio
features per
5      window; `compare_increment` then runs the SHIPPED harness between a shuttle+person
variant
6      and a shuttle+person+audio variant ? so audio is measured as a marginal contribution ON
TOP
7      of person, not against raw shuttle. Audio enters only as columns the learned scorer
weights
8      (no special-casing, default-OFF); the golden-set ?F1 decides whether it earns its keep.
9
10     Run (no GPU):
11         python -m backend.eval.fusion_audio --manifest output/human_lovo_manifest.json
--features-dir output --augment --compare
12     """
13
14     from __future__ import annotations
15
16     import argparse
17     import json
18     import logging
19     import os
20     from typing import Any, Dict, List
21
22     from backend.eval import experiment
23     from backend.eval.fusion_compare import delta_stats
24     from backend.eval.fusion_golden import (
25         PERSON_FEATURE_NAMES,
26         SHUTTLE_FEATURE_NAMES,
27         _derive_video_path,
28     )
29     from backend.pipeline.audio.hit_detector import detect_hits_in_video
30     from backend.pipeline.detectors import audio_features as af
31
32     logger = logging.getLogger(__name__)
33
34
35     def augment(manifest_path: str, features_dir: str, k: float = 2.5) -> List[str]:
36         """Add the 3 audio features to each existing ``<name>_golden_features.json``
window."""
37         with open(manifest_path, encoding="utf-8") as f:
38             specs = json.load(f)
39             written: List[str] = []
40             for spec in specs:
41                 jp = os.path.join(features_dir, f"{spec['name']}_golden_features.json")
42                 if not os.path.isfile(jp):
43                     print(
44                         f"[fusion_audio] {spec['name']}: no feature JSON ? run fusion_golden
first; skipping"
45                     )
46                     continue
47                 video = _derive_video_path(spec)
48                 print(
49                     f"[fusion_audio] {spec['name']}: detecting audio hits (k={k})...",
50                     flush=True,
51                 )
52                 hits = detect_hits_in_video(video, k=k)
53                 with open(jp, encoding="utf-8") as f:
54                     data = json.load(f)
55                     for c in data["candidates"]:
56                         c["audio"] = af.compute_audio_features(hits, c["start"], c["end"])

```

```

57         with open(jp, "w", encoding="utf-8") as f:
58             json.dump(data, f, indent=2)
59         print(
60             f"[fusion_audio] {spec['name']}: {len(hits)} hits over
{len(data['candidates'])} windows",
61             flush=True,
62         )
63         written.append(jp)
64     return written
65
66
67     def load_videos_with_audio(features_dir: str) -> List[experiment.VideoCandidates]:
68         """Load the FULL-FUSION-SCHEMA feature JSONs into VideoCandidates including the
audio
69         columns (requires `augment` to have run ? raises if a fusion JSON lacks audio).
70
71         Leaner velocity-windowing-only files (no ``shuttle``/``person`` cue dicts) are
SKIPPED, not
72         crashed on, so a mixed corpus loads the fusion subset cleanly (see
``_is_fusion_schema``).
73         The skip is checked BEFORE the audio-presence guard, so a genuine fusion file that
simply
74         hasn't been ``--augment``-ed still raises rather than being silently dropped."""
75         return experiment.load_golden_videos(
76             features_dir,
77             shuttle_columns=SHUTTLE_FEATURE_NAMES,
78             person_columns=PERSON_FEATURE_NAMES,
79             audio_columns=af.AUDIO_FEATURE_NAMES,
80             audio_policy="require",
81             skip_logger=logger,
82             skip_message="fusion_audio.load_videos_with_audio: skipping %s ? windowing-only
"
83             "schema (no shuttle/person cue dicts), not a fusion-feature file",
84             no_audio_error="{path} has no audio features ? run ``--augment`` first",
85         )
86
87
88     def compare_incremental(
89         videos: List[experiment.VideoCandidates], iou: float = 0.5, threshold: float = 0.5
90     ) -> Dict[str, Any]:
91         """Incremental audio lift: shuttle+person vs shuttle+person+audio (honest
LOVO)."""
92         person = experiment.Variant(
93             name="shuttle_person",
94             feature_names=list(SHUTTLE_FEATURE_NAMES) + list(PERSON_FEATURE_NAMES),
95             threshold=threshold,
96         )
97         person_audio = experiment.Variant(
98             name="shuttle_person_audio",
99             feature_names=list(SHUTTLE_FEATURE_NAMES)
100             + list(PERSON_FEATURE_NAMES)

```

25. assistant (2026-07-03T17:34:22.796Z)

I see the issue! Let me check line 37-38 more carefully:

26. user (2026-07-03T17:34:23.466Z)

```

1     """Fusion P4 ? measure the INCREMENTAL audio-cue lift on top of the person cue (CPU-
only).
2
3     Reuses the person-feature JSONs from `fusion_golden.py` (no GPU re-detect): `augment`
runs
4     `detect_hits_in_video` once per video (ffmpeg + numpy, CPU) and adds the 3 audio
features per

```

```

5 window; `compare_increment` then runs the SHIPPED harness between a shuttle+person
variant
6 and a shuttle+person+audio variant ? so audio is measured as a marginal contribution ON
TOP
7 of person, not against raw shuttle. Audio enters only as columns the learned scorer
weights
8 (no special-casing, default-OFF); the golden-set ?F1 decides whether it earns its keep.
9
10 Run (no GPU):
11     python -m backend.eval.fusion_audio --manifest output/human_lovo_manifest.json
--features-dir output --augment --compare
12     """
13
14     from __future__ import annotations
15
16     import argparse
17     import json
18     import logging
19     import os
20     from typing import Any, Dict, List
21
22     from backend.eval import experiment
23     from backend.eval.fusion_compare import delta_stats
24     from backend.eval.fusion_golden import (
25         PERSON_FEATURE_NAMES,
26         SHUTTLE_FEATURE_NAMES,
27         _derive_video_path,
28     )
29     from backend.pipeline.audio.hit_detector import detect_hits_in_video
30     from backend.pipeline.detectors import audio_features as af
31
32     logger = logging.getLogger(__name__)
33
34
35     def augment(manifest_path: str, features_dir: str, k: float = 2.5) -> List[str]:
36         """Add the 3 audio features to each existing ``<name>_golden_features.json``
window."""
37         with open(manifest_path, encoding="utf-8") as f:
38             specs = json.load(f)
39             written: List[str] = []
40             for spec in specs:
41                 jp = os.path.join(features_dir, f"{spec['name']}_golden_features.json")
42                 if not os.path.isfile(jp):
43                     print(
44                         f"[fusion_audio] {spec['name']}: no feature JSON ? run fusion_golden
first; skipping"
45                     )
46                     continue
47                 video = _derive_video_path(spec)
48                 print(
49                     f"[fusion_audio] {spec['name']}: detecting audio hits (k={k})...",
50                     flush=True,
51                 )
52                 hits = detect_hits_in_video(video, k=k)
53                 with open(jp, encoding="utf-8") as f:
54                     data = json.load(f)
55                     for c in data["candidates"]:
56                         c["audio"] = af.compute_audio_features(hits, c["start"], c["end"])
57                 with open(jp, "w", encoding="utf-8") as f:
58                     json.dump(data, f, indent=2)
59                 print(
60                     f"[fusion_audio] {spec['name']}: {len(hits)} hits over
{len(data['candidates'])} windows",
61                     flush=True,
62                 )

```

```

63         written.append(jp)
64     return written
65
66
67     def load_videos_with_audio(features_dir: str) -> List[experiment.VideoCandidates]:
68         """Load the FULL-FUSION-SCHEMA feature JSONs into VideoCandidates including the
audio
69         columns (requires `augment` to have run ? raises if a fusion JSON lacks audio).
70

```

27. assistant (2026-07-03T17:34:25.368Z)

Now let me check the experiment.py file:

28. user (2026-07-03T17:34:27.091Z)

```

246:         d = json.load(f)
713:         entries = json.load(f)

```

29. assistant (2026-07-03T17:34:30.191Z)

Let me check calibrate_local, distill_local, and rally_seq_proto to see how they call load_resolved_manifest:

30. user (2026-07-03T17:34:31.308Z)

```

diff --git a/backend/eval/calibrate_local.py b/backend/eval/calibrate_local.py
index 687f790..4734f38 100644
--- a/backend/eval/calibrate_local.py
+++ b/backend/eval/calibrate_local.py
@@ -24,7 +24,6 @@ where videos.json = [{"name", "trajectory", "fps", "frame_width", "labels"}, ...]
    from __future__ import annotations

    import argparse
    -import json
    from typing import Callable, Dict, List, Tuple

    from backend.eval.calibrate_wasb import load_golden_gts
@@ -34,6 +33,7 @@ from backend.eval.calibration import (
    leave_one_video_out,
    select_best,
)
+from backend.eval.golden_manifest import load_resolved_manifest
    from backend.pipeline.detectors import rally_gate
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

@@ -164,8 +164,7 @@ def main() -> None:
    ap.add_argument("--iou", type=float, default=0.5)
    ap.add_argument("--metric", default="f1", choices=["f1", "precision", "recall"])
    args = ap.parse_args()
    -    with open(args.manifest, encoding="utf-8") as f:
    -        specs = json.load(f)
    +    specs = load_resolved_manifest(args.manifest)
    run(specs, iou=args.iou, metric=args.metric)

```

31. user (2026-07-03T17:34:34.277Z)

```

diff --git a/backend/eval/distill_local.py b/backend/eval/distill_local.py
index 52ac85a..a05b3ff 100644
--- a/backend/eval/distill_local.py
+++ b/backend/eval/distill_local.py
@@ -24,13 +24,13 @@ Run:
    from __future__ import annotations

```

```

import argparse
import json
from typing import Dict, List, Optional, Tuple

from backend.eval import windowing as _windowing
from backend.eval.calibrate_wasb import load_golden_gts
from backend.eval.classifier import LogisticRegression
from backend.eval.gemini_refine import merge_overlaps
+from backend.eval.golden_manifest import load_resolved_manifest
from backend.eval.metrics import score
from backend.eval.training import label_candidates
from backend.pipeline.detectors import rally_gate
@@ -218,8 +218,7 @@ def main() -> None:
    )
    ap.add_argument("--model-out", default="output/local_rally_model.json")
    args = ap.parse_args()
    with open(args.manifest, encoding="utf-8") as f:
    -     specs = json.load(f)
    +     specs = load_resolved_manifest(args.manifest)
    run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)

```

32. user (2026-07-03T17:34:38.326Z)

```

diff --git a/backend/eval/rally_seq_proto.py b/backend/eval/rally_seq_proto.py
index 2ea9483..b017f94 100644
--- a/backend/eval/rally_seq_proto.py
+++ b/backend/eval/rally_seq_proto.py
@@ -29,7 +29,6 @@ Run:
    from __future__ import annotations

    import argparse
    import json
    from typing import Dict, List, Tuple

    import numpy as np
@@ -38,6 +37,7 @@ from scipy.stats import rankdata

    from backend.eval.calibrate_wasb import load_golden_gts
    from backend.eval.classifier import LogisticRegression
+from backend.eval.golden_manifest import load_resolved_manifest
    from backend.eval.metrics import score

    # Feature extraction is single-sourced in backend/features (ADR-008): training and
@@ -196,8 +196,7 @@ def main() -> None:
        help="JSON: [{name,trajectory,fps,frame_width,labels}, ...]",
    )
    args = ap.parse_args()
    with open(args.manifest, encoding="utf-8") as f:
    -     run(json.load(f))
    +     run(load_resolved_manifest(args.manifest))

    if __name__ == "__main__":

```

33. assistant (2026-07-03T17:34:40.369Z)

Now let me check the tests to see if they handle ManifestPathError:

34. user (2026-07-03T17:34:40.857Z)

```

1      """SERVED Gate-A regression litmus ? the *production* FusionSegmenter path on real
footage.
2
3      The flagged gap (#146): the served default flip (`default_segementer=fusion_hybrid` +

```

```

4     ``min_crossings=2``) is **mock-tested only** on real video (``test_fusion_hybrid.py``
drives a
5     MagicMock runner). The offline ablation litmus
6     (``test_ablation.test_litmus_reproduces_gate_a``)
7     validates the Gate-A *signal* ? but on the **permissive proposal windowing**
8     (``distill_local.PROPOSAL``), NOT the production operating point. This module closes the
gap: it
9     runs the real ``FusionSegmenter`` decision seams (``_enrich_candidate_stats`` net-
crossings +
10    ``_select_for_handoff`` Gate A) over windows from the **production**
11    ``trajectory_to_action_windows`` config, on the cached golden WASB trajectories, and
asserts
12    the served path's measured behaviour.
13    GPU-free / Gemini-free (person cue OFF) ? it stops at the AI-handoff boundary. Skips
cleanly in
14    CI where the golden trajectories are absent; runs on the owner box where they are
present, over
15    **whatever golden clips are currently on disk** (``_golden_specs`` requires >= 6).
16    CORPUS NOTE ? re-grounded 2026-06-22 (the golden set is mutable and grows over time;
this is the
17    SECOND re-grounding ? #236 pinned the 2026-06-18 9-clip snapshot, which the corpus has
since
18    outgrown). The original 6 *single-court* golden clips this litmus was first calibrated
on
19    (2026-06-14) have been retired from the box ? their labels are gone ? so the present
golden set is
20    the **15 multicourt clips** now on disk (mahadevpura_1/2/singles, GX0101xx, GX0x0094,
21    Badminton_BXH_2, Boxhill_Doubles, adarsh_avi_singles, kushagra_singles,
22    targetest_doubles, gbaaddy,
23    testlarge_short). On that corpus the served path is **recall-starved**: WASB tracks a
single shuttle
24    while several courts rally at once, so ``trajectory_to_action_windows`` emits FEWER
windows than
25    there are rallies (served over-seg ratio < 1, i.e. under-segmentation). Because Gate A
only ever
26    DROPS windows, on this under-segmented corpus it is mildly **F1-negative** (mean served
?F1 ?
27    -0.023), not the ~neutral -0.008 the original 6 showed. Under the permissive PROPOSAL
windowing the
28    same gate is ~F1-neutral (?F1 ? +0.03) while still cutting over-segmentation
(1.36?0.79).
29
30    The canonical **+0.074 / 6-of-6 / p=0.031** Gate-A win was a property of the
*original-6* corpus
31    and is NOT reproduced here (those clips are no longer on the box); it is preserved in
the archive
32    (``docs/archives/quality-iterations/2026-06-first-ab-experiment/``). These tests now pin
the
33    **present-corpus** behaviour so a future windowing/corpus change that silently shifts it
trips the
34    litmus. The pinned magnitudes are a date-stamped snapshot of the present golden set and
will need
35    re-measurement as the corpus grows.
36    ""
37
38    from __future__ import annotations
39
40    import os
41
42    import pytest
43
44    # cv2 is imported transitively by trajectory_hybrid (the served engine). On a box
without it

```

```

45     # (CI), skip the whole module rather than erroring at import.
46     pytest.importorskip("cv2")
47
48     from backend.eval import segmentation_metrics as _segm # noqa: E402
49     from backend.eval import served_gate_a as sga # noqa: E402
50     from backend.eval.calibrate_wasb import load_golden_gts # noqa: E402
51     from backend.eval.golden_manifest import ( # noqa: E402
52         ManifestPathError,
53         load_resolved_manifest,
54     )
55     from backend.eval.metrics import score # noqa: E402
56     from backend.pipeline.detectors import rally_gate # noqa: E402
57     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner # noqa: E402
58
59     # The golden manifest lives in the MAIN checkout's output/ (gitignored; absent in a
fresh
60     # worktree / CI). Resolve it relative to this repo root first, with an env override for
an
61     # out-of-tree checkout.
62     _REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
63     MANIFEST = os.environ.get(
64         "SERVED_GATE_A_MANIFEST",
65         os.path.join(_REPO_ROOT, "output", "human_lovo_manifest.json"),
66     )
67
68
69     def _golden_specs():
70         """Return rebased manifest specs when trajectory + label files are present, else
71         []."""
72         if not os.path.isfile(MANIFEST):
73             return []
74         try:
75             specs = load_resolved_manifest(MANIFEST)
76         except ManifestPathError:
77             return []
78         return specs if len(specs) >= 6 else []
79
80     _SPECS = _golden_specs()
81     _skip = pytest.mark.skipif(
82         not _SPECS,
83         reason=f"golden trajectories absent (manifest {MANIFEST!r}); runs on the owner box",
84     )
85
86
87     # ---- #398 regression: golden-free (runs in CI; only the golden DATA is absent there,
not cv2) ----
88     def test_served_idx_cfg_is_typed_and_seams_accept_it(tmp_path):
89         """The served litmus must pass a TYPED IndexingConfig (what production passes), not
a dict ?
90         `_enrich_candidate_stats` reads `idx_cfg.fusion`, so a dict raised AttributeError
(#398). Build
91         the config + drive the full served seam path on a synthetic trajectory (no golden
data needed)."""
92         from backend.config.models import IndexingConfig
93
94         cfg = sga._served_idx_cfg(2)
95         assert isinstance(cfg, IndexingConfig)
96         assert cfg.fusion.min_crossings == 2
97         traj = tmp_path / "synth.csv"
98         rows = ["Frame,Visibility,X,Y"]
99         for i in range(120): # oscillate across the net so windows + crossings form
100             x = 200 + (i % 20) * 40

```

Good, now let me check how fusion_audio is called from tests:

36. user (2026-07-03T17:34:44.575Z)

```
tests/test_audio_features.py:from backend.eval import fusion_audio
tests/test_audio_features.py:    shuttle = {fn: 1.0 for fn in
fusion_audio.SHUTTLE_FEATURE_NAMES}
tests/test_audio_features.py:    person = {fn: 1.0 for fn in
fusion_audio.PERSON_FEATURE_NAMES}
tests/test_audio_features.py:    videos = fusion_audio.load_videos_with_audio(d)
tests/test_audio_features.py:    res = fusion_audio.compare_increment(videos, iou=0.5)
tests/test_audio_features.py:    assert "audio" in fusion_audio.format_result(res, 0.5).lower()
tests/test_audio_features.py:    videos = fusion_audio.load_videos_with_audio(d)
tests/test_audio_features.py:        "shuttle": {fn: 0.0 for fn in
fusion_audio.SHUTTLE_FEATURE_NAMES},
tests/test_audio_features.py:        "person": {fn: 0.0 for fn in
fusion_audio.PERSON_FEATURE_NAMES},
tests/test_audio_features.py:        fusion_audio.load_videos_with_audio(str(tmp_path))
tests/test_experiment.py:    (ablation.load_videos / fusion_compare.load_videos /
fusion_audio.load_videos_with_audio)."""
tests/test_experiment.py:    - ``fusion_audio.load_videos_with_audio`` REQUIRES audio ?
raises ``SystemExit``;
tests/test_experiment.py:    from backend.eval import ablation, fusion_audio, fusion_compare
tests/test_experiment.py:    # fusion_audio: require ? raises SystemExit on the same audio-less
file.
tests/test_experiment.py:        fusion_audio.load_videos_with_audio(d)
```

37. assistant (2026-07-03T17:34:46.721Z)

Now let me check the promotion.py for promote_if_served_safe:

38. user (2026-07-03T17:34:47.936Z)

```
backend/eval/promotion.py:24:paths are untouched until a caller routes its A/B through
:func:`promote_if_served_safe`.
backend/eval/promotion.py:35:__all__ = ["ServedGateResult", "promote_if_served_safe",
"served_regresses"]
backend/eval/promotion.py:91:        *promotion-grade* scores (served by default ? see
:func:`promote_if_served_safe`).
backend/eval/promotion.py:117:def promote_if_served_safe(
backend/eval/serve_contrast.py:7::func:`backend.eval.promotion.promote_if_served_safe`.
backend/eval/serve_contrast.py:37:from backend.eval.promotion import promote_if_served_safe
backend/eval/serve_contrast.py:127:    decision = promote_if_served_safe(
```

39. assistant (2026-07-03T17:34:49.894Z)

Let me check the golden_manifest module guard for the smoke command:

40. user (2026-07-03T17:34:50.226Z)

```
375
376
377     if __name__ == "__main__":
378         raise SystemExit(main())
379
```

41. assistant (2026-07-03T17:34:52.340Z)

Good. Now let me check if any callers of load_golden in rally_seg_eval might catch exceptions or if there's any other issue with the strict=True default:

42. user (2026-07-03T17:34:52.761Z)

```
1     """Nightly END-TO-END engine regression ? reel-count + quality + cost (the GCP-VM
```



```

nightly).
2
3     Runs the **configured + checkpointed** pipeline (the real WASB GPU detector, as a
deployment serves
4     it) end-to-end on 2-3 short owned golden videos and asserts the **number of reels does
not change**
5     vs a frozen baseline. Unlike ``scripts/nightly_regression.py`` (PR #202 ? a CPU re-score
of CACHED
6     trajectories, the windowing/scorer only), this one runs the *full* pipeline on the *real
model*, so it
7     needs a GPU ? hence it runs on an ephemeral GCP GPU VM
(``deploy/gcp/nightly_regression/``), emails the
8     owner the report, and self-deletes. It is **default-additive**: a tripwire, not the
promotion gate.
9
10    What it asserts / reports, per the owner's ask ("quality metrics + regression report +
cost"):
11    ? REEL COUNT per video ? ``len(play_segments)`` after segmentation; the detected
highlight clips.
12    Pure-CV / WASB detection is deterministic for a fixed (video, config, weights), so
this is asserted
13    **EXACT** (with a re-run-once guard for the one known flake: a transient DB-insert-
None, see $guard).
14    ? QUALITY METRICS per video (when golden labels are provided) ? the R2 tolerance-match
block from
15    ``backend.eval.rally_seg_eval`` (tol_f1 / f1@0.5 / f1@0.25), compared with a small
tolerance.
16    ? COST of the run ? VM uptime (or processing wall) × machine $/hr via
``backend.billing`` ? USD + INR.
17
18    Extensible pattern (add a video = one manifest row + re-freeze):
19    ``eval_baselines/reelcount_manifest.json`` ? the videos (gs:// / local / gdrive://
URIs) + segmenter
20    ``eval_baselines/reelcount_baseline.json`` ? the frozen expected counts (regenerated,
never hand-edited)
21
22    Usage
23    -----
24    Freeze the baseline from the current configured+checkpointed engine (after an intended
change, or first run):
25    python scripts/nightly_reelcount.py --update-baseline
26
27    Nightly check (exit 1 on regression; writes output/nightly_reelcount/<date>.md; emails
if --email):
28    python scripts/nightly_reelcount.py --email
29
30    Exit codes: 0 pass · 1 regression (reel-count changed / a video errored / quality
dropped) · 2 operational
31    (manifest/videos missing, nothing ran) · 3 no baseline · 4 stale (corpus/segmenter
changed ? re-freeze).
32    ""
33
34    from __future__ import annotations
35
36    import argparse
37    import datetime as _dt
38    import json
39    import os
40    import sys
41    from dataclasses import dataclass, field
42    from typing import Any, Dict, List, Optional, Tuple
43
44    REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
45    if REPO_ROOT not in sys.path:
46        sys.path.insert(0, REPO_ROOT)

```

```
47
48     # billing is pure (typing only) ? safe to import at module top.
49     from backend import billing # noqa: E402
50
```

43. assistant (2026-07-03T17:34:54.910Z)

Let me check if nightly_reelcount uses rally_seg_eval.load_golden:

44. user (2026-07-03T17:34:56.482Z)

```
15:     ``backend.eval.rally_seg_eval`` (tol_f1 / f1@0.5 / f1@0.25), compared with a small
tolerance.
401:     from backend.eval import rally_seg_eval
509:         row = rally_seg_eval.score_intervals(intervals, gts)
```

45. assistant (2026-07-03T17:34:58.859Z)

Now let me check if there are any other calls to load_golden in the codebase:

46. user (2026-07-03T17:35:00.631Z)

```
backend/eval/ablation.py:462:     return experiment.load_golden_videos(
backend/eval/audio/e1_probe.py:9:Reuses: audio.hit_detector,
calibrate_wasb.load_golden_gts/make_predict_fn,
backend/eval/audio/e1_probe.py:19:from backend.eval.calibrate_wasb import BASELINE,
load_golden_gts, make_predict_fn
backend/eval/audio/e1_probe.py:67:     gts = load_golden_gts(labels_csv)
backend/eval/calibrate_local.py:15:Reuses: TrackNetRunner windowing, rally_gate,
calibrate_wasb.load_golden_gts,
backend/eval/calibrate_local.py:29:from backend.eval.calibrate_wasb import load_golden_gts
backend/eval/calibrate_local.py:36:from backend.eval.golden_manifest import
load_resolved_manifest
backend/eval/calibrate_local.py:91:         gts = load_golden_gts(s["labels"])
backend/eval/calibrate_local.py:167:     specs = load_resolved_manifest(args.manifest)
backend/eval/calibrate_wasb.py:41:def load_golden_gts(csv_path: str) -> List[Interval]:
backend/eval/calibrate_wasb.py:90:     gts = load_golden_gts(labels_csv)
backend/eval/distill_local.py:18:rally_gate, metrics, calibrate_wasb.load_golden_gts,
gemini_refine.merge_overlaps.
backend/eval/distill_local.py:30:from backend.eval.calibrate_wasb import load_golden_gts
backend/eval/distill_local.py:33:from backend.eval.golden_manifest import load_resolved_manifest
backend/eval/distill_local.py:117:     gts = load_golden_gts(spec["labels"])
backend/eval/distill_local.py:221:     specs = load_resolved_manifest(args.manifest)
backend/eval/eval_partial_labels.py:16: - calibrate_wasb.make_predict_fn / load_golden_gts /
BASELINE
backend/eval/eval_partial_labels.py:34:from backend.eval.calibrate_wasb import BASELINE,
load_golden_gts, make_predict_fn
backend/eval/eval_partial_labels.py:100:     gts_all = load_golden_gts(labels_csv)
backend/eval/eval_partial_labels.py:207:     gts_all = load_golden_gts(labels_csv)
backend/eval/experiment.py:207:def load_golden_videos(
backend/eval/export_wasb_segments.py:28:from backend.eval.calibrate_wasb import BASELINE,
load_golden_gts, make_predict_fn
backend/eval/export_wasb_segments.py:233:         gts = load_golden_gts(labels_csv)
backend/eval/fusion_audio.py:75:     return experiment.load_golden_videos(
backend/eval/fusion_compare.py:44:     return experiment.load_golden_videos(
backend/eval/fusion_golden.py:27:from backend.eval.calibrate_wasb import load_golden_gts
backend/eval/fusion_golden.py:28:from backend.eval.golden_manifest import load_resolved_manifest
backend/eval/fusion_golden.py:74:     gts = load_golden_gts(str(spec["labels"]))
backend/eval/fusion_golden.py:164:     specs = load_resolved_manifest(manifest_path)
backend/eval/golden_manifest.py:110:def load_resolved_manifest(
backend/eval/gt_loader.py:6:- ``calibrate_wasb.load_golden_gts``: DictReader requiring
``start_time/end_time``
backend/eval/rally_seg_eval.py:30:from backend.eval.golden_manifest import
load_resolved_manifest
```

```

backend/eval/rally_seg_eval.py:184:def load_golden(
backend/eval/rally_seg_eval.py:189:     manifest = load_resolved_manifest(manifest_path,
required_fields=("trajectory",))
backend/eval/rally_seg_eval.py:782:     golden = load_golden(args.manifest, args.features_dir)
backend/eval/rally_seq_proto.py:38:from backend.eval.calibrate_wasb import load_golden_gts
backend/eval/rally_seq_proto.py:40:from backend.eval.golden_manifest import
load_resolved_manifest
backend/eval/rally_seq_proto.py:129:     gts = load_golden_gts(s["labels"])
backend/eval/rally_seq_proto.py:199:     run(load_resolved_manifest(args.manifest))
backend/eval/served_gate_a.py:31:from backend.eval.calibrate_wasb import load_golden_gts
backend/eval/served_gate_a.py:32:from backend.eval.golden_manifest import load_resolved_manifest
backend/eval/served_gate_a.py:122:     gts = load_golden_gts(labels)
backend/eval/served_gate_a.py:199:     specs = load_resolved_manifest(manifest_path)
backend/eval/serve_contrast.py:33:from backend.eval.calibrate_wasb import load_golden_gts
backend/eval/serve_contrast.py:35:from backend.eval.golden_manifest import
load_resolved_manifest
backend/eval/serve_contrast.py:86:     gts = load_golden_gts(str(spec["labels"]))
backend/eval/serve_contrast.py:198:     specs = load_resolved_manifest(args.manifest)
docs/archives/quality-iterations/2026-06-18-lovo-resweep-n13/lovo_resweep.py:23:from
backend.eval.rally_seg_eval import load_golden, windows_for # noqa: E402
docs/archives/quality-iterations/2026-06-18-lovo-resweep-n13/lovo_resweep.py:72:     golden =
load_golden(MANIFEST, FEATURES)
scratch/ab_n6_lovo.py:12:from backend.eval.calibrate_wasb import load_golden_gts

```

47. assistant (2026-07-03T17:35:03.724Z)

Now let me specifically check the archived and scratch files to see if they would break:

48. user (2026-07-03T17:35:04.486Z)

```

1      """LOVO (leave-one-video-out) re-sweep of the milestone windowing thresholds on the
CURRENT
2      golden corpus. Answers: at this n, do the tuned values (cap=6 + R4) still hold, and
would
3      re-FITTING the cap per held-out fold beat just using the fixed value?
4
5      Honest by construction: the cap is chosen on the OTHER n-1 videos and scored on the
held-out
6      one (no train=test leakage). Metric = tIoU F1@0.5 (corpus headline) + R2 toIF1 (the
tuning metric).
7      Re-scoring only ? cached trajectories, no GPU.
8
9      Reproduce: add any new clip via scratch/ingest_golden.py, then run this. The sys.path
insert
10     below points at the worktree carrying the latest eval code ? adjust if your layout
differs.
11     """
12
13     import os
14     import statistics
15     import sys
16     from dataclasses import replace
17     from math import comb
18
19     MAIN = r"C:\Users\avidu\Projects\badminton-highlight-indexer"
20     sys.path.insert(0, os.path.join(MAIN, ".claude", "worktrees", "boxhill17"))
21
22     from backend.eval import metrics # noqa: E402
23     from backend.eval.rally_seg_eval import load_golden, windows_for # noqa: E402
24     from backend.eval.tolerance_metrics import match_1to1 # noqa: E402
25     from backend.eval.windowing import PRESETS # noqa: E402
26
27     MANIFEST = os.path.join(MAIN, "output", "human_lovo_manifest.json")
28     FEATURES = os.path.join(MAIN, "output")
29     CAPS = [0, 4, 6, 8, 12] # 0 = bounded-windowing OFF (the over-merge baseline)

```

```

30 R4 = dict(inactivity_end=1.5, inactivity_rally_thresh=0.20, inactivity_min_density=0.30)
31 SERVED = PRESETS["served"]
32
33
34 def mean(xs):
35     return statistics.mean(xs) if xs else 0.0
36
37
38 def preset(cap, r4=True):
39     kw = dict(max_window_duration=float(cap))
40     if r4:
41         kw.update(R4)
42     return replace(SERVED, **kw)
43
44
45 def f1_tiou(v, p):
46     return metrics.score(windows_for(v, p), v["gts"], 0.5).f1
47
48
49 def tolf1(v, p):
50     return match_1to1(windows_for(v, p), v["gts"], tau_start=2.0, tau_end=1.5)[3]
51
52
53 def sign_test(a, b):
54     """a,b: per-name dicts. Returns (wins_a, losses_a, ties, two-sided binomial p)."""
55     wins = losses = ties = 0
56     for k in a:
57         if a[k] - b[k] > 1e-9:
58             wins += 1
59         elif b[k] - a[k] > 1e-9:
60             losses += 1
61         else:
62             ties += 1
63     n = wins + losses
64     if n == 0:
65         return wins, losses, ties, 1.0
66     kk = min(wins, losses)
67     p = sum(comb(n, i) for i in range(0, kk + 1)) / (2**n) * 2
68     return wins, losses, ties, min(1.0, p)
69
70
71 def main():
72     golden = load_golden(MANIFEST, FEATURES)
73     names = [v["name"] for v in golden]
74     print(f"corpus n={len(golden)}\n")
75
76     for label, scorer in (("tIoU F1@0.5", f1_tiou), ("R2 tolF1", tolf1)):
77         pv = {c: {v["name"]: scorer(v, preset(c)) for v in golden} for c in CAPS}
78         glob = {c: mean([pv[c][n] for n in names]) for c in CAPS}
79         best_cap = max(CAPS, key=lambda c: glob[c])
80         print(f"=== {label} ? global cap sweep (R4 ON) ===")
81         for c in CAPS:
82             mark = " <- best" if c == best_cap else " (current)" if c == 6 else ""
83             print(f" cap={c if c else 'OFF':>3}: mean {glob[c]:.3f}{mark}")
84         # LOVO cap selection
85         lovo = []
86         for v in golden:
87             others = [n for n in names if n != v["name"]]
88             pick = max(CAPS, key=lambda c: mean([pv[c][o] for o in others]))
89             lovo.append((v["name"], pick, pv[pick][v["name"]]))
90         lovo_mean = mean([s for _, _, s in lovo])
91         picks = {}
92         for _, c, _ in lovo:
93             picks[c] = picks.get(c, 0) + 1
94         print(

```

```

95         f" LOVO (cap chosen per held-out fold): mean {lovo_mean:.3f} | fixed cap=6:
{glob[6]:.3f} "
96         f"| delta(LOVO-fixed6) {lovo_mean - glob[6]:+.3f}"
97     )
98     print(f" per-fold cap picks: {dict(sorted(picks.items()))}")
99     # R4 on vs off at cap=6
100    on = {v["name"]: scorer(v, preset(6, True)) for v in golden}

```

49. assistant (2026-07-03T17:35:07.372Z)

Now let me check if load_golden can raise ManifestPathError in this archived script. The issue is that the archived script at line 72 calls load_golden with a hard-coded manifest path, and if the trajectory field is missing in one of the rows, it will now raise an exception instead of silently continuing. Let me also check if training/gen0/harness.py has similar issues:

50. user (2026-07-03T17:35:09.080Z)

```

diff --git a/training/gen0/harness.py b/training/gen0/harness.py
index 6eec29f..433d2cb 100644
--- a/training/gen0/harness.py
+++ b/training/gen0/harness.py
@@ -36,6 +36,7 @@ from typing import Dict, List, Optional
import numpy as np

from backend.eval.classifier import LogisticRegression
+from backend.eval.golden_manifest import load_resolved_manifest
from backend.eval.gt_loader import load_gt_intervals
from backend.eval.rally_seq_proto import (
    _seg_f1,
@@ -217,8 +218,7 @@ def gate_verdict(eval_res: dict, floor: Optional[dict]) -> dict:
def run(
    manifest_path: str, out_dir: str, version: str, floor_path: Optional[str] = None
) -> dict:
-    with open(manifest_path, encoding="utf-8") as f:
-        specs = json.load(f)
+    specs = load_resolved_manifest(manifest_path)
    floor = None
    if floor_path:
        with open(floor_path, encoding="utf-8") as f:

```

51. assistant (2026-07-03T17:35:11.644Z)

Now let me check the repo_root() function to see if it has the correct parent count:

52. user (2026-07-03T17:35:13.140Z)

```
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

53. assistant (2026-07-03T17:35:15.670Z)

Good. Now let me check whether there's a scenario where nightly_regression or other callers would have stale manifest paths at fingerprint time vs score time. Let me look more closely at the fingerprint / baseline logic:

54. user (2026-07-03T17:35:15.996Z)

```

618    def run_corpus(
619        configs: List[NightlyConfig],
620        manifest: str,
621        features_dir: str,
622        iou: float = DEFAULT_IΟΥ,
623    ) -> Tuple[Dict[str, Any], Dict[str, List[str]]]:
624        """Score every tracked config over the golden corpus. Returns (snapshot, skipped-
per-config).

```

```

625
626     ``snapshot`` is the serialisable structure stored as / compared to the baseline.
627     ``skipped``
628     lists manifest videos NOT scored (missing trajectory or empty GT) per config ?
629     surfaced in
630     the report so the corpus can't silently shrink.
631     from backend.eval import rally_seg_eval
632
633     golden = rally_seg_eval.load_golden(manifest, features_dir)
634     manifest_names = [v["name"] for v in golden]
635     if not golden:
636         raise RuntimeError(f"no golden videos loaded from {manifest}")
637
638     snapshot: Dict[str, Any] = {
639         "manifest": _display_path(manifest),
640         "iou": iou,
641         "configs": {},
642     }
643     skipped: Dict[str, List[str]] = {}
644     scored_any = False
645     for cfg in configs:
646         result = rally_seg_eval.evaluate(
647             golden, cfg.preset, iou=iou, min_crossings=cfg.min_crossings
648         )
649         summary = summarize_run(result)
650         summary["note"] = cfg.note
651         snapshot["configs"][cfg.name] = summary
652         scored = set(summary["per_video"])
653         skipped[cfg.name] = [n for n in manifest_names if n not in scored]
654         scored_any = scored_any or bool(scored)
655     if not scored_any:
656         raise RuntimeError(
657             f"scored 0 videos for every config ? are the trajectory CSVs in {manifest}
658             present "
659             f"and do the {features_dir}/<name>_golden_features.json files have 'gts'?"
660         )
661     return snapshot, skipped
662
663 def load_baseline(path: str) -> Optional[Dict[str, Any]]:
664     if not os.path.exists(path):

```

55. assistant (2026-07-03T17:35:19.346Z)

Now let me check if `fusion_audio.augment()` calls `load_resolved_manifest` or if it directly loads JSON. Looking at the code again - it doesn't! Let me verify:

56. user (2026-07-03T17:35:19.708Z)

```

35     def augment(manifest_path: str, features_dir: str, k: float = 2.5) -> List[str]:
36         """Add the 3 audio features to each existing ``<name>_golden_features.json``
37         window."""
38         with open(manifest_path, encoding="utf-8") as f:
39             specs = json.load(f)
40             written: List[str] = []
41             for spec in specs:
42                 jp = os.path.join(features_dir, f"{spec['name']}_golden_features.json")
43                 if not os.path.isfile(jp):
44                     print(
45                         f"[fusion_audio] {spec['name']}: no feature JSON ? run fusion_golden
46                         first; skipping"
47                     )
48                     continue
49                 video = _derive_video_path(spec)
50                 print(

```

```

49         f"[fusion_audio] {spec['name']}: detecting audio hits (k={k})...",
50         flush=True,
51     )
52     hits = detect_hits_in_video(video, k=k)
53     with open(jp, encoding="utf-8") as f:
54         data = json.load(f)
55     for c in data["candidates"]:
56         c["audio"] = af.compute_audio_features(hits, c["start"], c["end"])
57     with open(jp, "w", encoding="utf-8") as f:
58         json.dump(data, f, indent=2)
59     print(
60         f"[fusion_audio] {spec['name']}: {len(hits)} hits over
{len(data['candidates'])} windows",
61         flush=True,
62     )
63     written.append(jp)
64     return written

```

57. assistant (2026-07-03T17:35:22.205Z)

This is a CRITICAL issue! `fusion\_audio.augment()` at line 37-38 still uses raw `json.load()` instead of `load\_resolved\_manifest()`. Let me check if fusion_audio.augment() is called from anywhere:

58. user (2026-07-03T17:35:23.778Z)

backend/eval/experiment.py:190: ``fusion\_audio.augment`` and older JSONs predate it).

59. user (2026-07-03T17:35:27.367Z)

```

backend/eval/fusion_audio.py:35: def augment(manifest_path: str, features_dir: str, k: float =
2.5) -> List[str]:
backend/eval/fusion_audio.py:145:         augment(args.manifest, args.features_dir, args.k)

```

60. user (2026-07-03T17:35:29.612Z)

```

130     ap.add_argument("--features-dir", default="output")
131     ap.add_argument("--k", type=float, default=2.5, help="hit-detector sensitivity")
132     ap.add_argument(
133         "--augment",
134         action="store_true",
135         help="add audio features to the existing JSONs",
136     )
137     ap.add_argument(
138         "--compare", action="store_true", help="report the incremental audio LOVO ?F1"
139     )
140     ap.add_argument("--iou", type=float, default=0.5)
141     ap.add_argument("--record", action="store_true")
142     args = ap.parse_args()
143
144     if args.augment:
145         augment(args.manifest, args.features_dir, args.k)
146     if args.compare:
147         videos = load_videos_with_audio(args.features_dir)
148         if len(videos) < 2:
149             raise SystemExit(f"need >=2 augmented feature files in {args.features_dir}")
150         res = compare_increment(videos, iou=args.iou)
151         print(format_result(res, args.iou))
152         if args.record:
153             row = experiment.record_experiment(res)
154             print(
155                 f"recorded -> eval_baselines/experiment_leaderboard.json
({row['timestamp']})"
156             )

```

```

157
158
159     if __name__ == "__main__":
160         main()
161

```

61. assistant (2026-07-03T17:35:31.947Z)

Now let me check if there's a potential mismatch scenario. Let me look at the docs mentioned in the PR angle:

62. user (2026-07-03T17:35:33.195Z)

```

diff --git a/docs/ALPHA_LAUNCH_READINESS.md b/docs/ALPHA_LAUNCH_READINESS.md
index 04d46d2..bcc7fea 100644
--- a/docs/ALPHA_LAUNCH_READINESS.md
+++ b/docs/ALPHA_LAUNCH_READINESS.md
@@ -96,7 +96,7 @@ Legend: TODO = not started, IN PROGRESS = actively being worked, DONE =
shipped/
|----|-----|-----|-----|-----|----|
| A0 | Create this alpha readiness tracker and index it in docs | `badminton-highlight-indexer`
| - | No | DONE | #451 |
| A1 | Stand up alpha serving endpoint: Cloud Run/GPU or approved host, health check, edge auth
ON, upload -> process -> jobs -> compile -> download verified | `khealsutra` + `badminton-
highlight-indexer` | A0 | Owner spend / CF config | TODO | - |
-| A2 | Make the n=15 golden manifest portable/reproducible; add a path/corpus smoke check that
fails loudly when labels or trajectories are missing | `badminton-highlight-indexer` | A0 | No |
TODO | - |
+| A2 | Make the n=15 golden manifest portable/reproducible; add a path/corpus smoke check that
fails loudly when labels or trajectories are missing | `badminton-highlight-indexer` | A0 | No |
IN PROGRESS | this PR |
| A3 | Promote the 15-video source-video/MD5 records into the collector/vault path; reconcile
collector's six-video source manifest with the 15-video eval corpus | `sports-data-collector` +
vault | A2 | Owner upload/storage scope | TODO | - |
| A4 | Refresh the n=15 nightly regression baseline intentionally and record the exact
command/output; keep stale-baseline warnings until reviewed | `badminton-highlight-indexer` | A2
| Reviewer sign-off | TODO | - |
| A5 | Recompute the heuristic n=15 floor and replace the stale `heuristic_lovo_n6` floor for
future Gen-0 comparisons | `badminton-highlight-indexer` | A2 | No | TODO | - |
@@ -118,6 +118,15 @@ Legend: TODO = not started, IN PROGRESS = actively being worked, DONE =
shipped/
5. **Ablation/fusion PRs:** fix ablation import, complete 15-video features, rerun
promotion/rejection decisions.
6. **Launch decision PR/docs update:** record ship-gate target, C3 status, alpha copy limits,
and launch checklist.

+A2 smoke command:
+
+```powershell
+python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json --min-count
15
+python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json --min-count
15 --write output/human_lovo_manifest.local.json
+```
+
+The checker resolves stale absolute paths against the repo's sibling `Annotation Setup` folders
and `output/`, then exits non-zero if any required `trajectory` or `labels` file is absent.
+
## 9. Open questions

1. **Owner:** Is the first alpha allowed to use manual/offline corpus promotion, or should
P10/P11 be reopened before any tester wave?

```

63. assistant (2026-07-03T17:35:37.159Z)

Let me now check if any tests might break due to the strict=True default when called without exception handling:

64. user (2026-07-03T17:35:38.428Z)

```
tests/test_calibrate_wasb.py:9:    load_golden_gts,
tests/test_calibrate_wasb.py:39: def test_load_golden_gts_skips_incomplete(tmp_path):
tests/test_calibrate_wasb.py:40:     gts = load_golden_gts(_write_golden(tmp_path / "g.csv"))
tests/test_experiment.py:24:     load_golden_videos,
tests/test_experiment.py:141: def test_load_golden_videos_rejects_unknown_audio_policy(tmp_path):
tests/test_experiment.py:145:     load_golden_videos(
tests/test_fusion_compare.py:164: monkeypatch.setattr(fusion_golden, "load_golden_gts",
lambda labels: [(0.0, 10.0)])
tests/test_golden_manifest.py:74:     gm.load_resolved_manifest(manifest,
repo_root_path=repo)
tests/test_gt_loader.py:14: from backend.eval.calibrate_wasb import load_golden_gts
tests/test_gt_loader.py:41:     "loader", [load_gt_intervals, load_annotations, load_golden_gts]
tests/test_gt_loader.py:49:     "loader", [load_gt_intervals, load_annotations, load_golden_gts]
tests/test_served_gate_a_regression.py:50: from backend.eval.calibrate_wasb import
load_golden_gts # noqa: E402
tests/test_served_gate_a_regression.py:53:     load_resolved_manifest,
tests/test_served_gate_a_regression.py:74:     specs = load_resolved_manifest(MANIFEST)
tests/test_served_gate_a_regression.py:199:     gts = load_golden_gts(s["labels"])
tests/test_serve_contrast.py:54:     from backend.eval.calibrate_wasb import load_golden_gts
tests/test_serve_contrast.py:56:     gts = load_golden_gts(spec["labels"])
```

65. assistant (2026-07-03T17:35:40.698Z)

Let me look at test_golden_manifest.py to see if they test the strict mode:

66. user (2026-07-03T17:35:40.803Z)

```
1     import json
2     from pathlib import Path
3
4     import pytest
5
6     from backend.eval import golden_manifest as gm
7
8
9     def _write(path: Path, body: str = "x\n") -> Path:
10         path.parent.mkdir(parents=True, exist_ok=True)
11         path.write_text(body, encoding="utf-8")
12         return path
13
14
15     def _write_manifest(repo: Path, rows: list[dict]) -> Path:
16         path = repo / "output" / "human_lovo_manifest.json"
17         path.parent.mkdir(parents=True, exist_ok=True)
18         path.write_text(json.dumps(rows), encoding="utf-8")
19         return path
20
21
22     def test_resolves_stale_absolute_paths_to_workspace_layout(tmp_path):
23         workspace = tmp_path / "workspace"
24         repo = workspace / "badminton-highlight-indexer"
25         repo.mkdir(parents=True)
26         traj = _write(workspace / "Annotation Setup" / "Trajectories" / "foo_traj.csv")
27         old_label = _write(
28             workspace / "Annotation Setup" / "Golden Labelled" / "Foo.rallies.csv"
29         )
30         output_label = _write(repo / "output" / "Bar.rallies.csv")
31         manifest = _write_manifest(
32             repo,
```

```

33         [
34             {
35                 "name": "foo",
36                 "trajectory": "C:/old/Annotation
Setup/Collect/Trajectories/foo_traj.csv",
37                 "labels": "C:/old/Annotation Setup/Collect/Golden
Labelled/Foo.rallies.csv",
38                 "fps": 30,
39                 "frame_width": 1920,
40             },
41             {
42                 "name": "bar",
43                 "trajectory": "C:/old/Annotation
Setup/Collect/Trajectories/foo_traj.csv",
44                 "labels": "C:/old/badminton-highlight-indexer/output/Bar.rallies.csv",
45                 "fps": 30,
46                 "frame_width": 1920,
47             },
48         ],
49     )
50
51     specs, issues = gm.resolve_manifest_paths(manifest, repo_root_path=repo)
52
53     assert issues == []
54     assert Path(specs[0]["trajectory"]) == traj.resolve()
55     assert Path(specs[0]["labels"]) == old_label.resolve()
56     assert Path(specs[1]["labels"]) == output_label.resolve()
57
58
59     def test_strict_loader_reports_missing_required_files(tmp_path):
60         repo = tmp_path / "repo"
61         repo.mkdir()
62         manifest = _write_manifest(
63             repo,
64             [
65                 {
66                     "name": "missing",
67                     "trajectory": "C:/old/missing_traj.csv",
68                     "labels": "C:/old/missing.rallies.csv",
69                 }
70             ],
71         )
72
73         with pytest.raises(gm.ManifestPathError) as excinfo:
74             gm.load_resolved_manifest(manifest, repo_root_path=repo)
75
76         assert len(excinfo.value.issues) == 2
77         assert "file not found" in str(excinfo.value)
78
79
80     def test_trajectory_only_resolution_leaves_label_optional(tmp_path):
81         workspace = tmp_path / "workspace"
82         repo = workspace / "badminton-highlight-indexer"
83         repo.mkdir(parents=True)
84         traj = _write(workspace / "Annotation Setup" / "Trajectories" / "foo.csv")
85         manifest = _write_manifest(
86             repo,
87             [
88                 {
89                     "name": "foo",
90                     "trajectory": "C:/old/Annotation Setup/Collect/Trajectories/foo.csv",
91                     "labels": "C:/old/missing.rallies.csv",
92                 }
93             ],
94         )

```

```

95
96     specs, issues = gm.resolve_manifest_paths(
97         manifest, repo_root_path=repo, required_fields=("trajectory",)
98     )
99
100     assert issues == []
101     assert Path(specs[0]["trajectory"]) == traj.resolve()
102     assert specs[0]["labels"] == "C:/old/missing.rallies.csv"
103
104
105 def test_cli_writes_resolved_manifest(tmp_path, capsys):
106     workspace = tmp_path / "workspace"
107     repo = workspace / "badminton-highlight-indexer"
108     repo.mkdir(parents=True)
109     _write(workspace / "Annotation Setup" / "Trajectories" / "foo.csv")
110     _write(repo / "output" / "Foo.rallies.csv")
111     manifest = _write_manifest(
112         repo,
113         [
114             {
115                 "name": "foo",
116                 "trajectory": "C:/old/Annotation Setup/Collect/Trajectories/foo.csv",
117                 "labels": "C:/old/badminton-highlight-indexer/output/Foo.rallies.csv",
118             }
119         ],
120     )
121     out = repo / "output" / "human_lovo_manifest.local.json"
122
123     rc = gm.main(
124         [
125             "--manifest",
126             str(manifest),
127             "--repo-root",
128             str(repo),
129             "--min-count",
130             "1",
131             "--write",
132             str(out),
133         ]
134     )
135
136     captured = capsys.readouterr()
137     assert rc == 0
138     assert "OK:" in captured.out
139     assert out.is_file()
140     resolved = json.loads(out.read_text(encoding="utf-8"))
141     assert Path(resolved[0]["trajectory"]).is_file()
142     assert Path(resolved[0]["labels"]).is_file()
143
144
145 def test_cli_fails_loudly_on_missing_file(tmp_path, capsys):
146     repo = tmp_path / "repo"
147     repo.mkdir()
148     manifest = _write_manifest(
149         repo,
150         [{"name": "foo", "trajectory": "missing.csv", "labels": "missing.rallies.csv"}],
151     )
152
153     rc = gm.main(["--manifest", str(manifest), "--repo-root", str(repo)])
154
155     captured = capsys.readouterr()
156     assert rc == 2
157     assert "golden manifest path check failed" in captured.err
158     assert "foo: trajectory" in captured.err
159

```

67. assistant (2026-07-03T17:35:43.991Z)

Good. Now let me check if there are any scenarios where tests might break. Let me look at test_rally_seg_eval to see if it has tests for load_golden:

68. user (2026-07-03T17:35:45.847Z)

```
66:def test_build_preset_temporal_density_default_on():
75:def test_build_preset_temporal_density_flag_forces_on():
86:def test_score_intervals_perfect():
94:def test_score_intervals_surfaces_over_merge():
104:def test_aggregate_means_across_videos_with_ci():
117:def test_aggregate_empty():
123:def test_windows_for_netcross_gate(tmp_path):
141:def test_windows_for_prominent_court_gate(tmp_path):
162:def test_evaluate_threads_min_crossings(tmp_path):
175:def test_compare_includes_over_seg_guard(tmp_path):
204:def test_filter_by_min_duration_passthrough_and_strip():
214:def test_score_intervals_min_duration_filters_both_preds_and_gts():
229:def test_court_type_for_lookup_and_manifest_override():
244:def test_stratified_report_groups_by_court_and_duration(tmp_path):
277:def test_stratified_excludes_videos_with_no_long_golden_rally(tmp_path):
```

69. assistant (2026-07-03T17:35:47.936Z)

Now let me look for if the exported paths could create an issue with strict=True at call sites. Let me check fusion_compare.py:

70. user (2026-07-03T17:35:48.976Z)

```
diff --git a/tests/test_fusion_compare.py b/tests/test_fusion_compare.py
index d3477c8..f042eea 100644
--- a/tests/test_fusion_compare.py
+++ b/tests/test_fusion_compare.py
@@ -203,6 +203,8 @@ def test_fusion_golden_run_is_resumable(tmp_path, monkeypatch):

    out_dir = str(tmp_path / "out")
    os.makedirs(out_dir)
+   for name in ("a.csv", "b.csv", "a.rallies.csv", "b.rallies.csv"):
+       (tmp_path / name).write_text("x\n", encoding="utf-8")
    manifest = [
        {
            "name": "alpha",
@@ -267,6 +269,8 @@ def test_fusion_golden_smallest_first(tmp_path, monkeypatch):
    os.makedirs(out_dir)
    (tmp_path / "big.csv").write_text("x" * 2000)
    (tmp_path / "small.csv").write_text("x" * 20)
+   (tmp_path / "b.rallies.csv").write_text("x\n", encoding="utf-8")
+   (tmp_path / "s.rallies.csv").write_text("x\n", encoding="utf-8")
    manifest = [
        {
            "name": "big",
```

71. assistant (2026-07-03T17:35:52.730Z)

This is important! The test fixtures had to be updated to include label files. Now let me search for any other potential issues. Let me check if there are any callers that might not have all required fields:

72. user (2026-07-03T17:35:53.876Z)

```
backend/eval/calibrate_local.py:from backend.eval.golden_manifest import load_resolved_manifest
backend/eval/calibrate_local.py:from backend.pipeline.detectors import rally_gate
backend/eval/calibrate_local.py:from backend.pipeline.detectors.tracknet_runner import
```

```

TrackNetRunner
backend/eval/calibrate_local.py-
backend/eval/calibrate_local.py-# (velocity_thresh, merge_gap, min_window_duration,
min_crossings)
backend/eval/calibrate_local.py-LocalConfig = Tuple[float, float, float, int]
backend/eval/calibrate_local.py:    specs = load_resolved_manifest(args.manifest)
backend/eval/calibrate_local.py-    run(specs, iou=args.iou, metric=args.metric)
backend/eval/calibrate_local.py-
backend/eval/calibrate_local.py-
backend/eval/calibrate_local.py-if __name__ == "__main__":
backend/eval/calibrate_local.py-    main()
backend/eval/distill_local.py:from backend.eval.golden_manifest import load_resolved_manifest
backend/eval/distill_local.py:from backend.eval.metrics import score
backend/eval/distill_local.py:from backend.eval.training import label_candidates
backend/eval/distill_local.py:from backend.pipeline.detectors import rally_gate
backend/eval/distill_local.py:from backend.pipeline.detectors.tracknet_runner import
TrackNetRunner
backend/eval/distill_local.py-
backend/eval/distill_local.py:    specs = load_resolved_manifest(args.manifest)
backend/eval/distill_local.py-    run(specs, iou=args.iou, threshold=args.threshold,
model_out=args.model_out)
backend/eval/distill_local.py-
backend/eval/distill_local.py-
backend/eval/distill_local.py-if __name__ == "__main__":
backend/eval/distill_local.py-    main()
backend/eval/fusion_golden.py:from backend.eval.golden_manifest import load_resolved_manifest
backend/eval/fusion_golden.py:from backend.eval.training import label_candidates
backend/eval/fusion_golden.py:from backend.pipeline.detectors import fusion_features as ff
backend/eval/fusion_golden.py:from backend.pipeline.detectors import rally_gate
backend/eval/fusion_golden.py:from backend.pipeline.detectors.tracknet_runner import
TrackNetRunner
backend/eval/fusion_golden.py:from backend.utils.run_telemetry import RunTelemetry
backend/eval/fusion_golden.py:    specs = load_resolved_manifest(manifest_path)
backend/eval/fusion_golden.py-    if smallest_first:
backend/eval/fusion_golden.py-        # Process small videos first for quick feedback (the slow
full match lands last);
backend/eval/fusion_golden.py-        # the LOVO result is order-independent. Proxy size by the
trajectory CSV (one row/frame).
backend/eval/fusion_golden.py-        specs = sorted(
            specs,
            key=lambda x: len(x['trajectory']),
            reverse=True)
backend/eval/golden_manifest.py:def load_resolved_manifest(
manifest_path: str | os.PathLike[str],
*,
repo_root_path: str | os.PathLike[str] | None = None,
extra_roots: Iterable[str | os.PathLike[str]] = (),
required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
) -> Tuple[Dict[str, Any], List[str]]:
backend/eval/rally_seg_eval.py:from backend.eval.golden_manifest import load_resolved_manifest
backend/eval/rally_seg_eval.py:from backend.eval.metrics import Interval
backend/eval/rally_seg_eval.py:from backend.eval.windowing import PRESETS, SERVED,
WindowingPreset
backend/eval/rally_seg_eval.py-
backend/eval/rally_seg_eval.py-DEFAULT_MANIFEST = os.path.join("output",
"human_lovo_manifest.json")
backend/eval/rally_seg_eval.py-DEFAULT_FEATURES_DIR = "output"
backend/eval/rally_seg_eval.py:    manifest = load_resolved_manifest(manifest_path,
required_fields=("trajectory",))
backend/eval/rally_seg_eval.py-    out: List[Dict[str, Any]] = []
backend/eval/rally_seg_eval.py-    for v in manifest:
backend/eval/rally_seg_eval.py-        feat = os.path.join(features_dir,
f"{v['name']}_golden_features.json")
backend/eval/rally_seg_eval.py-        gts: List[Interval] = []
backend/eval/rally_seg_eval.py-        if os.path.exists(feat):
backend/eval/rally_seg_eval.py-            gts = load_resolved_manifest(feat,
required_fields=required_fields)
backend/eval/rally_seq_proto.py:from backend.eval.golden_manifest import load_resolved_manifest
backend/eval/rally_seq_proto.py:from backend.eval.metrics import score
backend/eval/rally_seq_proto.py-

```

```

backend/eval/rally_seq_proto.py-# Feature extraction is single-sourced in backend/features
(ADR-008): training and
backend/eval/rally_seq_proto.py-# serving import the SAME code, so train == serve by
construction. Names re-exported
backend/eval/rally_seq_proto.py-# here for back-compat with existing imports/tests.
backend/eval/rally_seq_proto.py:    run(load_resolved_manifest(args.manifest))
backend/eval/rally_seq_proto.py-
backend/eval/rally_seq_proto.py-
backend/eval/rally_seq_proto.py-if __name__ == "__main__":
backend/eval/rally_seq_proto.py-    main()
backend/eval/serve_contrast.py:from backend.eval.golden_manifest import load_resolved_manifest
backend/eval/serve_contrast.py-from backend.eval.metrics import score
backend/eval/serve_contrast.py-from backend.eval.promotion import promote_if_served_safe
backend/eval/serve_contrast.py-from backend.pipeline.detectors import rally_gate
backend/eval/serve_contrast.py-from backend.pipeline.detectors.tracknet_runner import
TrackNetRunner
backend/eval/serve_contrast.py-
backend/eval/serve_contrast.py:    specs = load_resolved_manifest(args.manifest)
backend/eval/serve_contrast.py:    result = run_contrast(
backend/eval/serve_contrast.py:        specs,
backend/eval/serve_contrast.py:        iou=args.iou,
backend/eval/serve_contrast.py:        min_crossings=args.min_crossings,
backend/eval/serve_contrast.py:        min_n=args.min_n,
backend/eval/served_gate_a.py:from backend.eval.golden_manifest import load_resolved_manifest
backend/eval/served_gate_a.py-from backend.eval.metrics import Interval, Score, score
backend/eval/served_gate_a.py-from backend.pipeline.detectors.tracknet_runner import
TrackNetRunner
backend/eval/served_gate_a.py-from backend.pipeline.segmenters.fusion_hybrid import
FusionSegmenter
backend/eval/served_gate_a.py-
backend/eval/served_gate_a.py-# Production windowing operating point (config.json indexing
defaults ? the values
backend/eval/served_gate_a.py:    specs = load_resolved_manifest(manifest_path)
backend/eval/served_gate_a.py:    results = evaluate_manifest(specs, iou=iou)
backend/eval/served_gate_a.py:    return results, aggregate(results)
backend/eval/served_gate_a.py-
backend/eval/served_gate_a.py-
backend/eval/served_gate_a.py-def main(argv: Optional[List[str]] = None) -> None:

```

73. assistant (2026-07-03T17:35:56.535Z)

Now let me check if the error scenarios are properly documented. Let me scan for the actual issue by looking at whether any callers would fail when trying to run without the trajectories or labels. Let me check nightly_reelcount for any missing trajectory/label handling:

74. user (2026-07-03T17:35:58.182Z)

(Bash completed with no output)

75. assistant (2026-07-03T17:36:02.762Z)

So nightly_regression.py was NOT modified. This means it still uses the old pattern. Let me check if it indirectly calls load_resolved_manifest through rally_seg_eval.load_golden:

Looking back at my earlier examination, `nightly\_regression.py` calls `rally\_seg\_eval.load\_golden()` at line 631, which now calls `load\_resolved\_manifest(manifest\_path, required\_fields=("trajectory",))`. So when nightly_regression.py runs, it will:

1. Call `rally\_seg\_eval.load\_golden(manifest, features\_dir)` (line 631)
2. Which now calls `load\_resolved\_manifest(manifest\_path, required\_fields=("trajectory",))` with `strict=True` by default
3. If any trajectory is missing, it will raise `ManifestPathError`

This is a problem in CI or fresh clones where the manifest might exist but trajectory files don't. Let me check if there's explicit error handling for this:

76. user (2026-07-03T17:36:03.232Z)

```
750     if not os.path.exists(args.manifest):
751         log.error(
752             "manifest not found: %s ? the golden corpus must be present locally.",
753             args.manifest,
754         )
755         return 2
756
757     try:
758         snapshot, skipped = run_corpus(
759             tracked_configs(), args.manifest, args.features_dir, args.iou
760         )
761     except RuntimeError as e:
762         log.error("%s", e)
763         return 2
764     snapshot["git_commit"] = _git_commit()
765     snapshot["generated_at"] = date
766
767     if args.update_baseline:
768         save_baseline(args.baseline, snapshot, tols)
769         log.info("baseline snapshot written to %s", args.baseline)
770         for cfg, s in snapshot["configs"].items():
771             agg = s["aggregate"]
772             log.info(
773                 " %s: n=%s tolF1=%.3f f1@0.5=%.3f merge_rate=%.3f",
774                 cfg,
775                 s["n"],
776                 agg.get("tol_f1") or 0.0,
777                 agg.get("f1@0.5") or 0.0,
778                 agg.get("merge_rate") or 0.0,
779             )
```

77. assistant (2026-07-03T17:36:05.474Z)

The try/except at line 757 only catches `RuntimeError`, not `ManifestPathError`. Let me verify that `ManifestPathError` is a `RuntimeError`:

78. user (2026-07-03T17:36:05.882Z)

```
50     class ManifestPathError(RuntimeError):
51         """Raised when a strict manifest load finds missing required files."""
52
53         def __init__(self, manifest_path: str | os.PathLike[str], issues:
54             Sequence[ManifestIssue]):
55             self.manifest_path = os.fspath(manifest_path)
56             self.issues = list(issues)
57             detail = "\n".join(f" - {i.message()}" for i in self.issues[:12])
58             more = "" if len(self.issues) <= 12 else f"\n ... {len(self.issues) - 12} more"
59             super().__init__(
60                 f"golden manifest path check failed for {self.manifest_path}: "
61                 f"{len(self.issues)} issue(s)\n{detail}{more}"
62             )
63
64     def repo_root() -> Path:
```

79. assistant (2026-07-03T17:36:08.967Z)

Good! `ManifestPathError` is a subclass of `RuntimeError`, so it WILL be caught by the exception handler in `nightly\_regression.py`. So that's not an issue.

Now let me check the specific scenario with fusion_audio that doesn't use load_resolved_manifest:

80. user (2026-07-03T17:36:09.995Z)

```
from backend.pipeline.detectors import audio_features as af

logger = logging.getLogger(__name__)

def augment(manifest_path: str, features_dir: str, k: float = 2.5) -> List[str]:
    """Add the 3 audio features to each existing ``<name>_golden_features.json`` window."""
    with open(manifest_path, encoding="utf-8") as f:
        specs = json.load(f)
    written: List[str] = []
    for spec in specs:
        jp = os.path.join(features_dir, f"{spec['name']}_golden_features.json")
        if not os.path.isfile(jp):
            print(
                f"[fusion_audio] {spec['name']}: no feature JSON ? run fusion_golden first;
skipping"
            )
```

81. assistant (2026-07-03T17:36:14.467Z)

This is a CRITICAL miss. The `fusion\_audio.augment()` function still loads the raw manifest without path resolution. If `fusion\_golden.run()` now resolves paths with `load\_resolved\_manifest()` and writes them back, then `fusion\_audio.augment()` reading the RAW manifest could:

1. See a rebased path that `fusion\_golden` wrote
2. Try to dereference it using raw file operations without path rebasing
3. Miss files in the expected search locations

But wait - looking at the PR diff more carefully, `fusion\_golden.run()` doesn't write the resolved manifest back, it just reads it. Let me verify:

82. user (2026-07-03T17:36:18.642Z)

<tool_use_error>InputValidationError: Read failed due to the following issue:
The parameter `offset` type is expected as `number` but provided as `array`</tool_use_error>

83. user (2026-07-03T17:36:21.514Z)

```
155
156     def run(
157         manifest_path: str,
158         out_dir: str,
159         frame_skip: int = 3,
160         iou: float = 0.5,
161         fresh: bool = False,
162         smallest_first: bool = False,
163     ) -> List[str]:
164         specs = load_resolved_manifest(manifest_path)
165         if smallest_first:
166             # Process small videos first for quick feedback (the slow full match lands
167             # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
168             # row/frame).
169             specs = sorted(
170                 specs,
171                 key=lambda s: (
172                     os.path.getsize(s["trajectory"])
173                     if os.path.isfile(s.get("trajectory", ""))
```



```

173             else 0
174         ),
175     )
176     os.makedirs(out_dir, exist_ok=True)
177     # Run-telemetry black box: a long GPU run can be OS-killed with no traceback, so
keep a tiny
178     # rotating runlog of velocity + machine health for the post-mortem. Constructing it
must never
179     # break extraction, so guard the build and fall back to no telemetry on any error.
180     telem: Optional[RunTelemetry]
181     try:
182         telem = RunTelemetry(
183             os.path.join(out_dir, "run_telemetry.jsonl"), label="fusion_golden"
184         )

```

84. assistant (2026-07-03T17:36:25.757Z)

So `fusion_golden.run()` reads the resolved specs but doesn't write them back to the manifest. This means if `fusion_audio.augment()` is called, it will read the ORIGINAL manifest with potentially stale paths. Let me trace the workflow:

1. User has old manifest with absolute paths (e.g., ``C:/old/Trajectories/foo.csv``)
2. ``fusion_golden.run()`` is called, which uses ``load_resolved_manifest()`` to rebase those paths to current locations
3. ``fusion_audio.augment()`` is called, which uses raw ``json.load()`` on the SAME manifest
4. ``augment()`` tries to use ``_derive_video_path(spec)`` from the stale paths

Let me check ``_derive_video_path``:

85. user (2026-07-03T17:36:26.727Z)

```

def _derive_video_path(spec: Dict[str, Any]) -> str:
    """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
golden labels path (`X.rallies.csv` -> `X.MP4`, the golden-set naming convention)."""
    if spec.get("video_path"):
        return str(spec["video_path"])
    labels = str(spec["labels"])
    if labels.endswith(".rallies.csv"):
        return labels[: -len(".rallies.csv")] + ".MP4"
    raise ValueError(
        f"{spec.get('name')}: no video_path and labels {labels!r} isn't *.rallies.csv"
    )
--
video_path = _derive_video_path(spec)

if detector is None:
    from backend.pipeline.detectors.person_detector import PersonDetector

    detector = PersonDetector({"indexing": {"use_gpu": True}})

# Single-pass person detection over ALL candidate windows (open the video ONCE, read

```

86. assistant (2026-07-03T17:36:32.769Z)

So ``_derive_video_path()`` takes the ``labels`` field from the spec and derives a video path. If ``fusion_audio.augment()`` gets a spec with a stale path in the ``labels`` field, it will try to construct a video path from an old absolute path. This might work if the file still exists at that location, but it's brittle.

However, the actual BREAKING scenario is more subtle. Let me think about when `fusion_audio` is actually called in a typical workflow. Looking at the module docstring:

```

...

```

Reuses the person-feature JSONs from ``fusion_golden.py`` (no GPU re-detect): ``augment`` runs

```
`detect_hits_in_video` once per video (ffmpeg + numpy, CPU) and adds the 3 audio features per window
` ``
```

So the workflow is:

1. ``fusion_golden.run()`` ? creates ``<name>_golden_features.json`` with person cues
2. ``fusion_audio.augment()`` ? adds audio features to those JSONs
3. ``fusion_audio.load_videos_with_audio()`` ? loads the augmented JSONs

The problem is in step 2: if the manifest has stale paths and they don't exist locally, ``augment()`` will fail when trying to call ``detect_hits_in_video(video, k=k)`` because ``video`` is constructed from a stale path.

Now let me write up the findings. Let me make one more check on where `fusion_audio` is called:

87. user (2026-07-03T17:36:33.813Z)

```
grep: Unmatched ( or \(
```

88. user (2026-07-03T17:36:37.376Z)

```
backend/eval/fusion_audio.py:35: def augment(manifest_path: str, features_dir: str, k: float = 2.5) -> List[str]:
backend/eval/fusion_audio.py:145:         augment(args.manifest, args.features_dir, args.k)
```

89. assistant (2026-07-03T17:36:41.897Z)

So ``augment()`` is called only from the CLI (``main()``) of `fusion_audio`. This means it's only an issue if someone runs:

```
` ``bash
python -m backend.eval.fusion_audio --manifest <manifest> --augment
` ``
```

Now let me create the final list of issues:

1. ****fusion_audio.augment()** - line 37-38**: Still uses raw ``json.load()`` instead of ``load_resolved_manifest()``. If the manifest has stale paths that don't exist locally, ``_derive_video_path()`` will try to use those stale paths for ``detect_hits_in_video()``, which will fail.

2. ****Scenario****: Fresh clone or CI where manifest exists but trajectory/label files don't. Running ``python -m backend.eval.fusion_audio --manifest output/human_lovo_manifest.json --augment`` will fail silently or with unclear errors.

Let me now check if there are any other manifest consumers I missed. Let me search for `"json.load"` in eval directory more carefully:

90. user (2026-07-03T17:36:43.030Z)

```
backend/eval/classifier.py:123:         return cls.from_dict(json.load(f))
backend/eval/experiment.py:246:         d = json.load(f)
backend/eval/experiment.py:713:         entries = json.load(f)
backend/eval/fusion_audio.py:37:         with open(manifest_path, encoding="utf-8") as f:
backend/eval/fusion_audio.py:38:             specs = json.load(f)
backend/eval/fusion_audio.py:54:             data = json.load(f)
backend/eval/golden_fixtures.py:743:         return json.load(f)
backend/eval/golden_manifest.py:75:         data = json.load(f)
backend/eval/per_user_regression.py:147:         return json.load(f)
backend/eval/rally_seg_eval.py:197:         (float(s), float(e)) for s, e in
(json.load(fh).get("gts") or [])
backend/eval/regression.py:75:         return json.load(f)
backend/eval/splits.py:41:         with open(manifest, "r", encoding="utf-8-sig") as f:
backend/eval/splits.py:42:             raw = json.load(f)
```

91. assistant (2026-07-03T17:36:45.946Z)

Let me check splits.py to see if it's a manifest consumer:

92. user (2026-07-03T17:36:46.142Z)

```
1      """Train/eval split guard (pure-logic core, in-container).
2
3      Spec from BACKLOG.md "Train/eval separation guardrails" / DEFERRED #13
4      and plan M0:
5      - Split unit = match (never clip / file MD5).
6      - load_split(manifest) -> (train_ids, eval_ids) keyed by match id.
7      - assert_disjoint(train_ids, eval_ids) raises on any overlap.
8      - This would have caught train/eval leakage in training/eval paths.
9      - Cross-repo collector manifest `split` field wiring + wiring into
10         distill_local / run_regression deferred to later PR.
11
12      Usage (once wired):
13         train, eval_ = load_split(manifest)
14         assert_disjoint(train, eval_)
15      """
16
17      from __future__ import annotations
18
19      import json
20      import os
21      from typing import Any, Dict, Iterable, List, Set, Tuple, Union, cast
22
23
24      def load_split(
25          manifest: Union[str, Iterable[Dict[str, Any]], os.PathLike[str]],
26      ) -> Tuple[Set[str], Set[str]]:
27          """Load (train_ids, eval_ids) sets keyed by match id.
28
29          manifest: path to JSON (str/bytes/PathLike) or iterable of dicts. Each entry should
30          have 'match_id'
31          (or 'id' fallback; must be non-empty str for real match key; name/video_id
32          deliberately not used as they are weak/MD5).
33
34          Raises ValueError on:
35          - non-list manifest after load
36          - no real match-level key (empty/falsy after explicit check)
37          - split label not exactly "train" or "eval"
38
39          The collector 'split' field and cross-repo wiring is deferred; this supports
40          synthetic or pre-split manifests for the pure-logic guard and tests.
41          """
42          if isinstance(manifest, (str, bytes, os.PathLike)):
43              with open(manifest, "r", encoding="utf-8-sig") as f:
44                  raw = json.load(f)
45                  if not isinstance(raw, list):
46                      raise ValueError(f"manifest must be a list of entries, got {type(raw)}")
47                  data = cast(List[Dict[str, Any]], raw)
48          else:
49              if isinstance(manifest, dict):
50                  raise ValueError(
51                      f"manifest must be a list of entries, got {type(manifest)}"
52                  )
53              data = list(manifest)
54              if not isinstance(data, list):
55                  raise ValueError(f"manifest must be a list of entries, got {type(data)}")
56
57          train_ids: Set[str] = set()
58          eval_ids: Set[str] = set()
```

```

58         for entry in data:
59             if not isinstance(entry, dict):
60                 continue
61             # Explicit not-None + non-empty for real match key; no silent fallback to
name/video_id (weak/MD5)
62             mid = entry.get("match_id")
63             if mid is None or str(mid).strip() == "":
64                 mid = entry.get("id")
65             if mid is None or str(mid).strip() == "":
66                 raise ValueError(
67                     f"no real match-level key (match_id or id) in entry: {entry}"
68                 )
69             mid = str(mid).strip()
70             sp = str(entry.get("split", "")).strip().lower()
71             if sp not in ("train", "eval"):
72                 raise ValueError(
73                     f"split label must be exactly 'train' or 'eval', got {sp!r} in entry:
{entry}"
74                 )
75             if sp == "eval":
76                 eval_ids.add(mid)
77             else:
78                 train_ids.add(mid)
79
80         return train_ids, eval_ids

```

93. assistant (2026-07-03T17:36:49.004Z)

This is a different kind of manifest (train/eval split), not the golden manifest. Not relevant.

Now let me verify my findings are complete by checking if there are any other entry points that call `load_golden` or `load_resolved_manifest` that I might have missed:

94. user (2026-07-03T17:36:50.916Z)

```

backend/eval/fusion_audio.py:125:def main() -> None:
backend/eval/fusion_golden.py:233:def main() -> None:
backend/eval/serve_contrast.py:174:def main() -> None:
backend/eval/served_gate_a.py:204:def main(argv: Optional[List[str]] = None) -> None:
backend/eval/rally_seq_proto.py:191:def main() -> None:
backend/eval/distill_local.py:206:def main() -> None:
backend/eval/calibrate_local.py:155:def main() -> None:

```

95. assistant (2026-07-03T17:36:57.283Z)

Perfect. Now let me compile my final findings. Based on my analysis, I found one critical issue:

****Issue:** ``fusion_audio.augment()`` uses raw ``json.load()`` without path resolution**

```

- **File**: `backend/eval/fusion_audio.py`
- **Lines**: 37-38
- **Summary**: The `augment()` function loads the manifest with raw `json.load()` instead of
`load_resolved_manifest()`. If manifest paths are stale (absolute paths to old locations), the
function will fail when trying to derive video paths for `detect_hits_in_video()`.
- **Failure scenario**: Running `python -m backend.eval.fusion_audio --manifest
output/human_lovo_manifest.json --augment` on a fresh clone or after moving the workspace will
fail because `_derive_video_path()` will try to use stale paths that don't exist locally. The
error will be unclear (from ffmpeg or video reading, not from missing trajectory files).

```

All other callers appear to properly use ``load_resolved_manifest()`` or handle the manifest correctly.

```

```json
[

```

```
{
 "file": "backend/eval/fusion_audio.py",
 "line": 37,
 "summary": "augment() loads manifest with raw json.load() instead of
load_resolved_manifest(), missing stale path resolution. If trajectory/label paths in manifest
are absolute (e.g., C:/old/...), _derive_video_path() will fail when trying to detect hits in a
nonexistent video file.",
 "failure_scenario": "User runs 'python -m backend.eval.fusion_audio --manifest
output/human_lovo_manifest.json --augment' after workspace move or in CI where manifest exists
but trajectory files don't. Fails at detect_hits_in_video() with unclear ffmpeg error, not
manifest error."
}
]
...
```